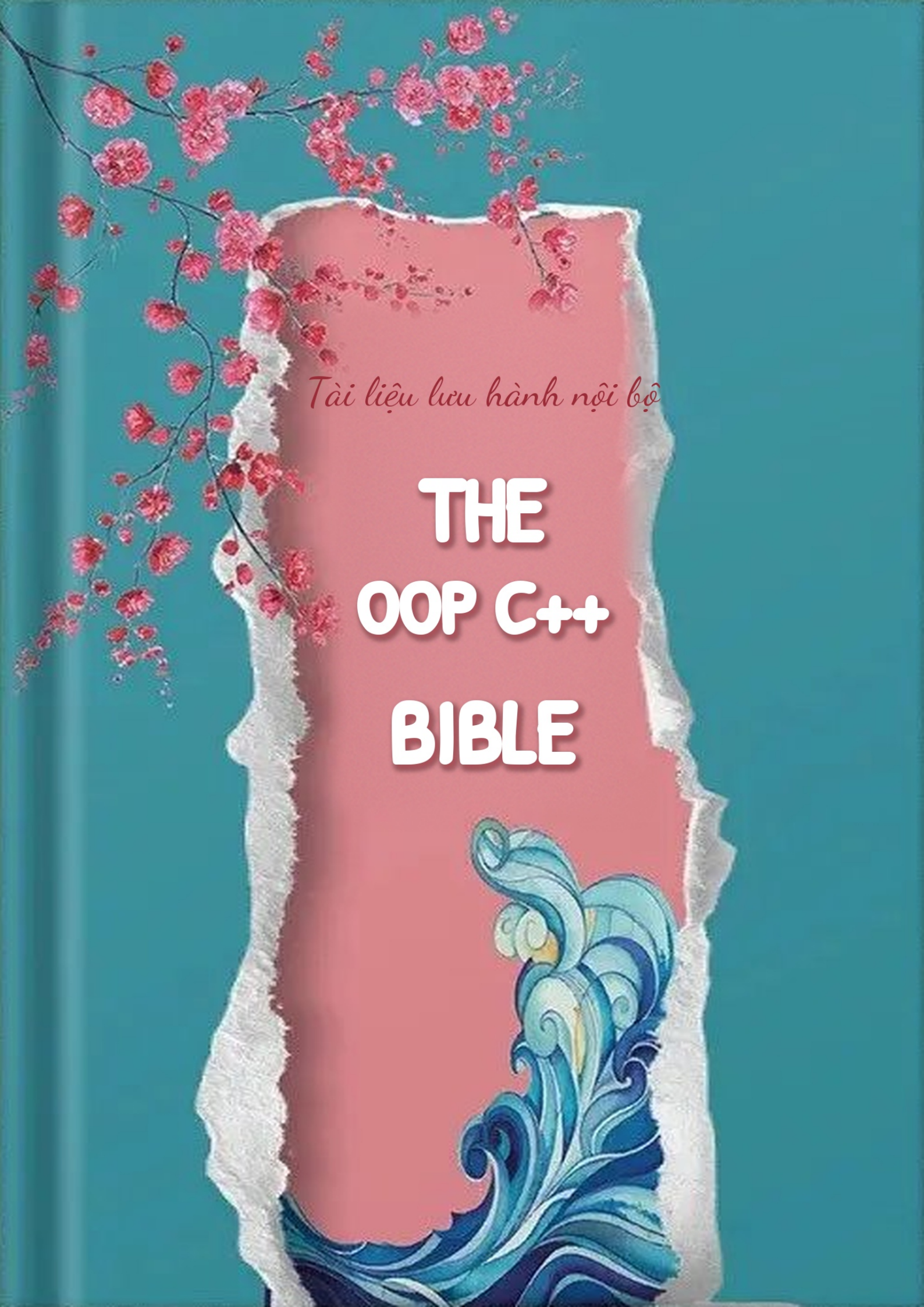

Tài liệu lưu hành nội bộ
THE
OOP C++
BIBLE

THE OOP C++ BIBLE

Giáo Trình Khoa Học Máy Tính & Kiến Trúc Phần Mềm Hướng Đối Tượng

---

Phân Tích Chuyên Sâu C++11/14/17 Dưới Tầng Phần Cứng, Quản Trị Bộ Nhớ RAM, VTable & Cơ Chế Trình Biên Dịch

---

Chuyên Luận Đào Tạo Kỹ Sư Công Nghệ Thông Tin Cao Cấp

Tái hiện toàn diện cơ sở lý thuyết, mô hình kiến trúc bộ nhớ, các hệ hình lập trình, và giải thuật tối ưu hóa tài nguyên chuẩn mực quốc tế ISO/IEC 14882.

Tủ Sách Kỹ Thuật Máy Tính & Khoa Học Phần Mềm

Lưu Hành Học Thuật & Nghiên Cứu Chuyên Sâu — Năm 2026

Contents

1 Triết lý Hệ hình & Không gian Bộ nhớ Máy tính (Philosophy & Memory Model) 1

1.1 Ma Trận So Sánh Kiến Trúc: C vs C++ 1
1.2 Kiến Trúc Bộ Nhớ Tiến Trình (Process Memory Layout) 2
1.3 Quản Lý Bộ Nhớ Động: So Sánh Đối Kháng malloc/free và new/delete 3
1.4 Con Trỏ, Tham Chiếu và Các Toán Tử Ép Kiểu Chuẩn Mực 4
1.4.1 Bản Chất Dưới Tầng Hợp Ngữ: Tham Chiếu Lvalue (Lvalue Reference) 4
1.4.2 Hệ Thống Bốn Toán Tử Ép Kiểu An Toàn (C++ Named Casts) 5

2 Lớp, Đối tượng & Cơ chế Khởi tạo Thực thể (Classes & Objects) 7

2.1 Tính Đóng Gói (Encapsulation) & Bộ Chỉ Định Quyền Truy Cập . . . 7
2.1.1 Cơ Chế Phân Quyền Truy Cập 7
2.1.2 Khảo Sát Bộ Nhớ: Sự Căn Chỉnh Dữ Liệu (Memory Alignment & Padding) 8
2.2 Bản Chất Con Trỏ Bản Thể (this Pointer) Dưới Tầng RAM & ABI Gọi Hàm 9
2.3 Vòng Đời Khởi Tạo Đối Tượng (Constructors Architecture) 10
2.3.1 Member Initializer List vs Phép Gán Trong Thân Constructor . 10
2.3.2 Hàm Tạo Ủy Quyền & Khởi Tạo Thành Viên Mặc Định (C++11) 11
2.3.3 Ngăn Chặn Ép Kiểu Ngầm Định Với Từ Khóa `explicit` 11
2.4 Hàm Hủy (Destructor) & Triết Lý Thiết Kế RAII 12
2.5 Thành Viên Static, Const và Kỹ Thuật Hàm Bạn (Friend) 12
2.5.1 Thành Viên Tĩnh (Static Data & Methods) 12
2.5.2 Từ Khóa `mutable` — Ngoại Lệ Trong Phương Thức Hằng . . . 13
2.5.3 Quan Hệ Bạn Bè (Friendship Architecture) 13

3 Quản trị Bộ nhớ, Move Semantics & Nạp chồng Toán tử (Memory & Overloading) 14

3.1 Sao Chép Nông (Shallow Copy) vs Sao Chép Sâu (Deep Copy) 14
3.1.1 Cơ Chế Phá Hủy Bộ Nhớ Của Shallow Copy (Double-Free Hazard) 14
3.1.2 Giải Pháp Chuẩn Mực: Sao Chép Sâu (Deep Copy) 15
3.2 Hệ Thống Phân Loại Giá Trị & Ngữ Nghĩa Di Dời (Move Semantics) . 16
3.2.1 Phân Loại Giá Trị Chuẩn Quốc Tế ISO/IEC 14882 (Value Categories) 16

3.2.2 Tham Chiếu Rvalue (`T&&`) & Ngữ Nghĩa Di Dời (Move Semantics) 17
3.3 Tiến Hóa Bộ Quy Tắc Quản Trị Tài Nguyên: Rule of 3, 5, 0 18
3.3.1 Quy Tắc Bộ Ba Cổ Điển (The Rule of Three — C++98) 18
3.3.2 Quy Tắc Bộ Năm Hiện Đại (The Rule of Five — C++11) . . . 18
3.3.3 Thành Ngữ Sao Chép và Hoán Vị (The Copy-and-Swap Idiom) 19
3.3.4 Quy Tắc Số Không (The Rule of Zero — Modern C++) 20
3.4 Nạp Chồng Toán Tử Toàn Diện (Operator Overloading Deep Dive) . . 20
3.4.1 Toán Tử Tiền Tố (++x) vs Hậu Tố (x++) 20
3.4.2 Nạp Chồng new/delete Cấp Lớp (Class-specific Allocators) & Placement New 20
3.5 Kiến Trúc Con Trỏ Thông Minh Chuẩn C++11 (Smart Pointers Architecture) 21
3.5.1 `std::unique_ptr<T>` — Quyền Sở Hữu Độc Quyền (Exclusive Ownership) 21
3.5.2 `std::shared_ptr<T>` — Quyền Sở Hữu Chia Sẻ (Shared Ownership) 21
3.5.3 `std::weak_ptr<T>` — Phá Vỡ Vòng Lặp Tham Chiếu (Circular Reference) 22

4 Kế thừa, Bố cục Bộ nhớ Nhị phân & Mô hình hóa UML (Inheritance) 23
4.1 Bản Chất Quan Hệ Hướng Đối Tượng & Ký Hiệu UML 23
4.1.1 Bố Cục Bộ Nhớ Đối Tượng Kế Thừa Đơn (Single Inheritance Memory Layout) 24
4.2 Ma Trận Biến Đổi Quyền Truy Cập Trong Kế Thừa 24
4.3 Vòng Đời Khởi Tạo và Hủy Kế Thừa 25
4.4 Hiện Tượng Che Khuất Tên (Name Hiding) vs Nạp Chồng 26
4.5 Cắt Xén Đối Tượng Đa Hình (Polymorphic Object Slicing) 27
4.6 Đa Kế Thừa, Bài Toán Kim Cương & Kế Thừa Ảo 28
4.6.1 Mô Hình Bài Toán Kim Cương (The Diamond Problem) 28

5 Tính Đa hình, Liên kết Động & Kiểu Trả về Đồng biến (Polymorphism) 30
5.1 Triết lý Đa hình & Cơ chế Liên kết (Binding) 30
5.2 Hàm Ảo, `override` và `final` trong C++11 31
5.2.1 Cơ chế Hàm Ảo (`virtual`) 31
5.2.2 Bộ Đôi Từ Khóa Ngữ Cảnh C++11: `override` và `final` 32
5.3 Phân tích Dưới Gầm Máy: Bảng Ảo (VTable) & Con trỏ vptr 32
5.3.1 Hàm Thunk Bù Độ Lệch Con Trỏ Bản Thể (Adjustor Thunk) . 33
5.4 Kiểu Trả Về Đồng Biến (Covariant Return Types) 34
5.5 Destructor Ảo (Virtual Destructor) & Bẫy Gọi Hàm Ảo Trong Constructor 34
5.5.1 Destructor Ảo — Nguyên Lý Sống Còn Trong Kế Thừa 34
5.5.2 Bẫy Gọi Hàm Ảo Trong Constructor & Destructor 35

6 Khuôn mẫu & Lập trình Tổng quát (Templates & Generic Programming) 36
6.1 Khuôn Mẫu Hàm & Khuôn Mẫu Lớp (Function & Class Templates) . . 36
6.2 Cơ Chế Phân Giải Tên Hai Giai Đoạn (Two-Phase Name Lookup) . . . 37

6.2.1 Từ Khóa `typename` và `.template` Cho Tên Phụ Thuộc (Dependent Names) . . . 37
6.3 Chuyên Biệt Hóa Khuôn Mẫu & Giải Pháp Hiện Đại C++17 . . . 37
6.3.1 Chuyên Biệt Hóa Toàn Phần (Full Specialization) . . . 37
6.3.2 C++17 `if constexpr`: Chấm Dứt Bẫy Nạp Chồng Herb Sutter 38
6.4 Các Tính Năng Khuôn Mẫu Hiện Đại: Variadics & Fold Expressions . 38
6.4.1 Biểu Thức Gập C++17 (Fold Expressions) . . . 38
6.5 CRTP: Đa Hình Tĩnh (Static Polymorphism) . . . 39

7 Xử lý Ngoại lệ & Kiến trúc Thư viện Chuẩn STL (Exceptions & STL) 40
7.1 Cơ Chế Bắt Ngoại Lệ & Tránh Cắt Xén Đối Tượng . . . 40
7.2 Trải Cuộn Ngăn Xếp (Stack Unwinding) & Liên Minh RAII . . . 41
7.3 Đặc Tả `noexcept` & Tối Ưu Hóa Tái Cấp Phát `std::vector` . . . 42
7.4 Ba Cấp Độ Cam Kết An Toàn Ngoại Lệ (Exception Safety Guarantees) 42
7.5 Kiến Trúc Tách Rời Trong Thư Viện Chuẩn C++ (STL) . . . 43
7.5.1 Kiến Trúc Ba Tầng Tách Rời (Decoupled Architecture) . . . 43
7.5.2 Kỹ Thuật Tag Dispatching Tại Compile-Time . . . 43
7.5.3 Bộ Cấp Phát Bộ Nhớ Tùy Biến (Allocators & Rebind Mechanism) 44

8 Nguyên lý Thiết kế SOLID & Mẫu Kiến trúc Hiện đại (SOLID & Design Patterns) 45
8.1 Hệ Thống Năm Nguyên Lý Thiết Kế SOLID . . . 45
8.1.1 1. Single Responsibility Principle (Nguyên Lý Đơn Trách Nhiệm - SRP) . . . 45
8.1.2 2. Open/Closed Principle (Nguyên Lý Đóng/Mở - OCP) . . . 46
8.1.3 3. Liskov Substitution Principle (Nguyên Lý Thay Thế Liskov - LSP) . . . 46
8.1.4 4. Interface Segregation Principle (Nguyên Lý Phân Tách Giao Diện - ISP) . . . 47
8.1.5 5. Dependency Inversion Principle (Nguyên Lý Đảo Ngược Phụ Thuộc - DIP) . . . 47
8.2 Các Mẫu Thiết Kế GoF Hiện Đại Hóa Trong C++11/14/17 . . . 47
8.2.1 Meyers Singleton (Thread-Safe Static Local Initialization) . . . 47
8.2.2 Mẫu Nhà Máy (Factory Method Pattern) Kết Hợp Smart Pointers 48
8.2.3 Mẫu Chiến Lược (Strategy Pattern): Đa Hình Động vs Đa Hình Tĩnh . . . 48

Lời Kết: Con Đường Làm Chủ C++ Hiện Đại 50

Tài Liệu Tham Khảo (Bibliography) 51

CHƯƠNG 1

Triết lý Hệ hình & Không gian Bộ nhớ Máy tính (Philosophy & Memory Model)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Sự Dịch Chuyển Hệ Hình (Paradigm Shift)

Lập trình Hướng Đối tượng (*Object-Oriented Programming — OOP*) trong C++ không đơn thuần là sự bổ sung cú pháp từ khóa `class` vào ngôn ngữ C. Đây là một cuộc cách mạng về **Hệ hình Tư duy Trừu tượng hóa (Abstraction Paradigm)**:

- **Ngôn ngữ C (Hệ hình Hướng Thủ tục — Procedural Paradigm):** Tiếp cận thế giới thực dưới góc nhìn của *Thuật toán và Luồng dữ liệu* (*Algorithms + Data Structures = Programs*). Dữ liệu là các cấu trúc thụ động (`struct`), trong khi các hàm nghiệp vụ thao tác tự do trên dữ liệu đó. Trạng thái toàn vẹn của dữ liệu không có bất kỳ cơ chế tự bảo vệ nào ở cấp độ biên dịch.
- **Ngôn ngữ C++ (Hệ hình Đa thức — Multi-Paradigm):** Tiếp cận bài toán thông qua việc mô hình hóa các *Thực thể Tự trị (Autonomous Entities)*. Một đối tượng là sự kết hợp hữu cơ không thể tách rời giữa **Trạng thái (Dữ liệu thành viên)** và **Hành vi (Phương thức thành viên)**. Tính toàn vẹn của đối tượng được bảo đảm tuyệt đối thông qua cơ chế đóng gói và kiểm soát quyền truy cập.

1.1 Ma Trận So Sánh Kiến Trúc: C vs C++

Bảng đối chiếu dưới đây tổng hợp các khác biệt bản thể luận giữa ngôn ngữ C và C++:

Table 1.1: Ma trận so sánh các đặc tính kiến trúc giữa C và C++

Đặc Tính Kiến Trúc	**Ngôn Ngữ C (C99/C11)**	**Ngôn Ngữ C++ (C++11/14/17)**
Hệ hình chủ đạo	Lập trình Thủ tục / Hướng chức năng	Đa hệ hình: Hướng đối tượng, Tổng quát, Hàm
Cơ chế kiểm soát truy cập	Hoàn toàn không có (Toàn bộ là public)	Ba tầng: `public`, `protected`, `private`
Cấp phát bộ nhớ động	Hàm thư viện `malloc()`, `free()`	Toán tử ngôn ngữ `new`, `delete`, Smart Pointers
Quản trị vòng đời đối tượng	Thủ công, dễ rò rỉ bộ nhớ	Tự động hóa qua Constructor/Destructor (RAII)
Tính đa hình	Con trỏ hàm thủ công trong struct	Bảng phương thức ảo tĩnh `vtable` và con trỏ `vptr`
Xử lý lỗi hệ thống	Mã lỗi trả về (*Error Codes*), biến `errno`	Cơ chế ném và bắt ngoại lệ (*Exceptions*), `noexcept`
Không gian tên	Toàn cục phẳng (*Flat namespace*), dễ xung đột	Phân cấp mô-đun qua `namespace`
Khả năng mở rộng toán tử	Không hỗ trợ	Nạp chồng toán tử toàn diện (*Operator Overloading*)

1.2 Kiến Trúc Bộ Nhớ Tiến Trình (Process Memory Layout)

Để nắm vững bản chất của mô hình đối tượng C++, kỹ sư phần mềm phải thấu suốt cách thức hệ điều hành và bộ vi xử lý tổ chức không gian bộ nhớ ảo (*Virtual Address Space*). Trên kiến trúc x86-64 với cơ chế phân trang 48-bit tiêu chuẩn, không gian địa chỉ 256 TB được chia tách thành hai dải địa chỉ chuẩn tắc (*Canonical Ranges*) 128 TB, phân cách bởi một khoảng trống không định tuyến (*Non-canonical Addressing Gap*) rộng 16 Exabytes:

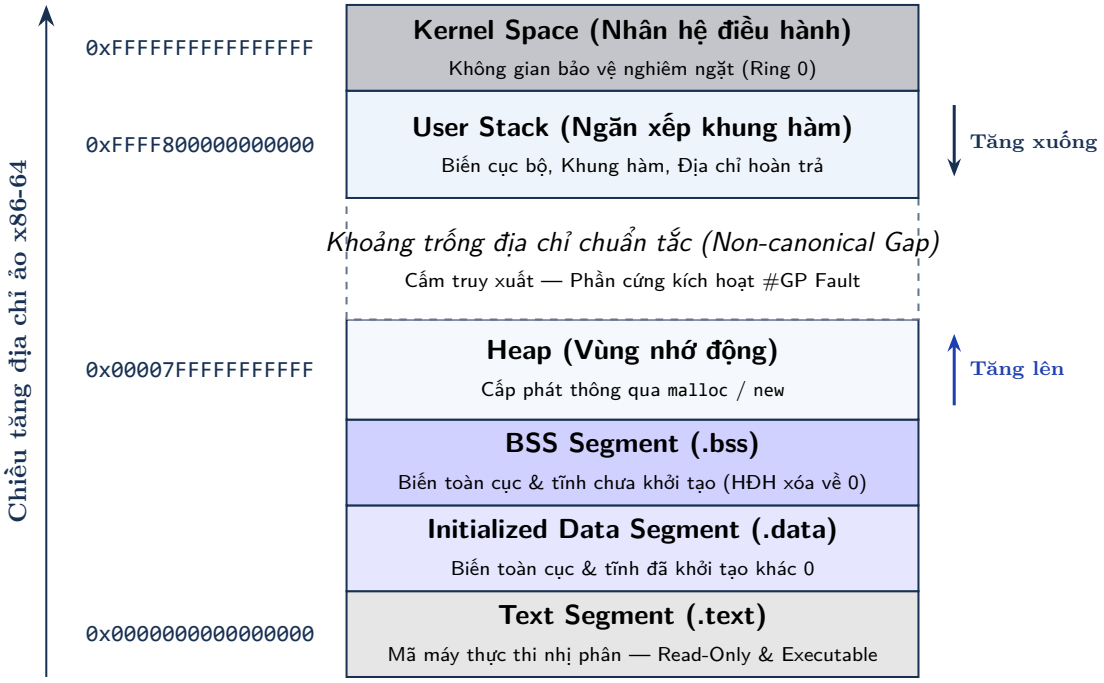


Figure 1.1: Kiến trúc phân tầng không gian địa chỉ ảo của tiến trình người dùng trong kiến trúc x86-64.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Giải Phẫu Chi Tiết Các Phân Vùng Bộ Nhớ

1. **Text/Code Segment:** Chứa mã máy nhị phân của tất cả các hàm thành viên và thuật toán. Phân vùng này được nạp cờ bảo vệ **Read-Only và Executable** (`RX`) từ bảng trang MMU để ngăn ngừa mã độc ghi đè lên

lệnh thực thi.

2. **Data Segment (`.data`) & BSS Segment (`.bss`):**
   - `.data`: Chứa các biến toàn cục (*global variables*) và biến tĩnh (*static variables*) đã được khởi tạo giá trị khác 0 tại thời điểm biên dịch.
   - `.bss` (*Block Started by Symbol*): Chứa các biến tĩnh/toàn cục chưa khởi tạo hoặc khởi tạo bằng 0. Khi nạp tiến trình, kernel chỉ cần cấp phát kích thước mà không tốn dung lượng lưu trữ trên đĩa cứng, sau đó ánh xạ về các trang bộ nhớ vật lý được xóa trắng về 0.
3. **Heap Segment:** Bộ nhớ động được điều phối bởi Trình quản lý bộ nhớ (*Memory Allocator*) của C/C++ runtime. Vùng nhớ này tăng trưởng theo chiều địa chỉ tăng dần thông qua các lời gọi hệ thống `brk()` hoặc `mmap()`.
4. **Stack Segment:** Vùng nhớ quản lý các khung hàm (*Stack Frames*). Khi một hàm được gọi, một khung ngăn xếp mới được cấp phát; khi hàm trả về, đỉnh ngăn xếp tự động co rút thông qua con trỏ thanh ghi `RSP`. Ngăn xếp tăng trưởng từ địa chỉ cao xuống địa chỉ thấp. Vị trí cơ sở của Stack được ngẫu nhiên hóa tại runtime bởi cơ chế ASLR (*Address Space Layout Randomization*) nhằm vô hiệu hóa các cuộc tấn công tràn bộ đệm.

1.3 Quản Lý Bộ Nhớ Động: So Sánh Đối Kháng malloc/free và new/delete

Sự khác biệt cốt tử giữa việc quản lý bộ nhớ trong C và C++ nằm ở sự hiểu biết về **Vòng đời của một Thực thể (Object Lifecycle)**.

```cpp
#include <iostream>
#include <cstdlib>

class SocketConnection {
private:
    int socketFd;
    char* networkBuffer;
public:
    SocketConnection(int port) : socketFd(port), networkBuffer(new char[1024]) {
        std::cout << "[Hardware] Socket initialized on port " << socketFd << "\n";
    }

    ~SocketConnection() {
        delete[] networkBuffer;
        std::cout << "[Hardware] Socket closed and buffer destroyed\n";
    }
};

void architecturalComparison() {
    // 1. C-STYLE ALLOCATION:
    // malloc() ONLY allocates raw bytes on Heap. It NEVER calls the Constructor!
    SocketConnection* pC = (SocketConnection*)malloc(sizeof(SocketConnection));

    // free() ONLY releases raw bytes. It NEVER calls the Destructor!
    // Result: networkBuffer is leaked forever in RAM!
    free(pC);

```

```
    // 2. C++ PROPER ALLOCATION:
    // Operator new allocates memory AND executes SocketConnection::
    SocketConnection(8080)
    SocketConnection* pCpp = new SocketConnection(8080);

    // Operator delete executes ~SocketConnection() AND releases memory
    delete pCpp;
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Điều Kiện Tồn Tại Của Tiền Tố Phân Bổ Mảng (Array Cookie) Theo Chuẩn Itanium ABI

Khi toán tử `new[]` cấp phát một mảng đối tượng:

```
SocketConnection* array = new SocketConnection[10];
```

Trình biên dịch thực hiện các thao tác mức máy sau:

1. **Quy tắc điều kiện Itanium C++ ABI (§2.7):** Trình biên dịch **CHỈ chèn thêm Array Cookie** (kích thước thường là 8 bytes trên kiến trúc 64-bit) khi và chỉ khi kiểu dữ liệu có **Hàm hủy phi tầm thường (Non-trivial Destructor)** hoặc lớp khai báo toán tử giải phóng chấp nhận tham số kích thước. Đối với các kiểu nguyên thủy (`int`, `double`) hoặc POD struct, trình biên dịch **hoàn toàn KHÔNG cấp phát cookie**.
2. **Ghi nhận số lượng phần tử:** Với lớp `SocketConnection` (có hàm hủy người dùng tự định nghĩa), trình biên dịch ghi số nguyên `10` vào 8 bytes đầu tiên của khối bộ nhớ được cấp phát trên Heap:

$$\text{Total Bytes} = \text{sizeof(size_t)} + 10 \times \text{sizeof(SocketConnection)}$$

3. **Dịch chuyển con trỏ:** Con trỏ trả về cho chương trình `array` được dịch chuyển qua khỏi Array Cookie (trỏ thẳng vào phần tử đầu tiên).
4. **Hậu quả khi gọi sai toán tử (`delete array` thay vì `delete[] array`):** Trình biên dịch sẽ đối xử với `array` như một đối tượng đơn lẻ. Nó chỉ gọi Destructor đúng 1 lần cho phần tử đầu tiên, bỏ qua 9 phần tử còn lại (thất thoát toàn bộ tài nguyên nội tại). Sau đó, nó truyền địa chỉ lệch 8 bytes cho hàm thu hồi bộ nhớ của hệ điều hành, làm hỏng cấu trúc siêu dữ liệu của Heap (**Heap Corruption**) và vi phạm điều khoản [expr.delete] p2 của Tiêu chuẩn Quốc tế ISO/IEC 14882.

1.4 Con Trỏ, Tham Chiếu và Các Toán Tử Ép Kiểu Chuẩn Mực

1.4.1 Bản Chất Dưới Tầng Hợp Ngữ: Tham Chiếu Lvalue (Lvalue Reference)

Trong ngôn ngữ C, mọi tham số truyền vào hàm mặc định là truyền theo giá trị (*Pass-by-value*). Để hàm có thể biến đổi giá trị bên ngoài, lập trình viên buộc phải sử dụng con trỏ (*Pointers*). Chuẩn C++ giới thiệu khái niệm **Tham chiếu (References)**,

đóng vai trò là một định danh bí danh (*Alias*) cho một vùng nhớ đã tồn tại.

```cpp
void incrementPointer(int* ptr) {
    if (ptr != nullptr) {
        *ptr += 1;
    }
}

void incrementReference(int& ref) {
    ref += 1; // Direct syntax without explicit dereferencing
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Bản Chất Của Tham Chiếu Dưới Mã Máy Hợp Ngữ (Assembly)

Nhiều tài liệu dẫn giải rằng tham chiếu "không chiếm bộ nhớ". Dưới lăng kính kiến trúc máy tính, nhận định này chỉ đúng khi trình biên dịch thực hiện tối ưu hóa nội dòng (*Inlining*).
Khi một tham chiếu được truyền qua ranh giới của một hàm không được inline, **MÃ MÁY HỢP NGỮ SINH RA HOÀN TOÀN TRÙNG KHỚP VỚI CON TRỎ HẰNG**:

```
; Call incrementReference(val):
lea rax, [rbp-4]        ; Load dia chi bien val vao thanh ghi RAX
mov rdi, rax            ; Chuyen dia chi vao RDI (tham so 1 theo System V
    AMD64 ABI)
call incrementReference ; Goi ham

; Ben trong ham incrementReference(int& ref):
mov rax, rdi            ; Nhan dia chi truyen vao
mov edx, [rax]          ; Giai tham chieu: doc gia tri tu dia chi
add edx, 1              ; Tang gia tri
mov [rax], edx          ; Ghi nguoc lai bo nho
```

Sự khác biệt cốt tử giữa Con trỏ và Tham chiếu:

1. Tham chiếu bắt buộc phải được khởi tạo ngay khi khai báo. Không tồn tại khái niệm "tham chiếu rỗng" (*Null Reference*) trong ngữ nghĩa hợp lệ của C++.
2. Tham chiếu không thể tái gán để trỏ sang một đối tượng khác sau khi đã khởi tạo (*Immutable Binding*). Mọi phép gán `ref = val2` thực chất là ghi đè giá trị lên đối tượng mà `ref` đang đại diện.

1.4.2 Hệ Thống Bốn Toán Tử Ép Kiểu An Toàn (C++ Named Casts)

C++ kiên quyết loại bỏ kiểu ép thô lỗ kiểu C (`(TargetType)expression`) vì nó che giấu các lỗi suy thoái con trỏ nguy hiểm. Thay vào đó, C++ cung cấp 4 toán tử chuyển đổi kiểu tường minh:

1. **`static_cast<T>(expr)`:** Chuyển đổi giữa các kiểu dữ liệu liên quan tại compile-time (ép kiểu số nguyên, số thực, chuyển đổi con trỏ trong cùng cây kế thừa khi đã chắc chắn về kiểu).

2. **`const_cast<T>(expr)`:** Thêm hoặc gỡ bỏ thuộc tính `const` hoặc `volatile` của biến. Đây là toán tử duy nhất trong C++ có năng lực gỡ bỏ tính chất hằng.
3. **`dynamic_cast<T>(expr)`:** Thực hiện ép kiểu an toàn trong cây phái sinh đa hình tại runtime thông qua bảng RTTI (*Run-Time Type Information*). Nếu thất bại trên con trỏ, nó trả về `nullptr`; nếu thất bại trên tham chiếu, nó ném ra ngoại lệ `std::bad_cast`.
4. **`reinterpret_cast<T>(expr)`:** Ép kiểu thô mức bit giữa các kiểu con trỏ không liên quan (ví dụ: chuyển đổi con trỏ thành số nguyên địa chỉ `uintptr_t`). Hoàn toàn không sinh mã máy biến đổi dữ liệu, chỉ thay đổi góc nhìn của trình biên dịch đối với chuỗi bit bộ nhớ.

CHƯƠNG 2

Lớp, Đối tượng & Cơ chế Khởi tạo Thực thể (Classes & Objects)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa Pháp Lập: Lớp (Class) và Đối Tượng (Object)

Lớp (Class) là một khuôn mẫu trừu tượng do người dùng định nghĩa, đóng vai trò là một kiểu dữ liệu mới (*User-Defined Type*), quy định cấu trúc bộ nhớ và các phép toán hợp lệ trên kiểu dữ liệu đó.
Đối tượng (Object) là một thực thể vật lý cụ thể (*Instance*) được cấp phát trong không gian bộ nhớ (trên Stack hoặc Heap), chiếm một vùng kích thước xác định và sở hữu trạng thái dữ liệu độc lập.

2.1 Tính Đóng Gói (Encapsulation) & Bộ Chỉ Định Quyền Truy Cập

2.1.1 Cơ Chế Phân Quyền Truy Cập

Tính đóng gói (*Encapsulation*) trong C++ thực hiện việc che giấu cấu trúc dữ liệu nội tại và bảo vệ trạng thái bất biến (*Class Invariants*) của đối tượng thông qua ba từ khóa chỉ định phạm vi truy cập:

- **`public`:** Các thành viên có thể được truy cập tự do từ bất kỳ điểm nào trong chương trình thông qua biến đối tượng, con trỏ hoặc tham chiếu.
- **`private`:** Các thành viên bị khóa hoàn toàn; chỉ có thể được truy cập bởi các hàm thành viên của chính lớp đó hoặc bởi các hàm/lớp được cấp quyền bạn bè (`friend`).
- **`protected`:** Tương tự như `private`, nhưng cho phép các lớp con cháu trong cây phái sinh (*Derived Classes*) có quyền truy cập trực tiếp.

Sự khác biệt cốt tử giữa `class` và `struct`: Trong C++, sự khác biệt duy nhất giữa `class` và `struct` nằm ở **Quyền truy cập mặc định**:

- Trong `struct`, mọi thành viên và mối quan hệ kế thừa mặc định là **`public`**.
- Trong `class`, mọi thành viên và mối quan hệ kế thừa mặc định là **`private`**.

2.1.2 Khảo Sát Bộ Nhớ: Sự Căn Chỉnh Dữ Liệu (Memory Alignment & Padding)

Kích thước của một đối tượng trong RAM không đơn thuần là tổng kích thước của các biến thành viên:

```
struct UnalignedStruct {
    char a;     // 1 byte
    // 3 bytes padding do kien truc 32-bit/64-bit yeu cau can le 4 bytes
    int b;      // 4 bytes
    char c;     // 1 byte
    // 7 bytes padding do kien truc 64-bit yeu cau can le 8 bytes
    double d;   // 8 bytes
}; // sizeof(UnalignedStruct) = 24 bytes (thay vi 1 + 4 + 1 + 8 = 14 bytes)!

struct OptimizedStruct {
    double d;   // 8 bytes (Offset 0)
    int b;      // 4 bytes (Offset 8)
    char a;     // 1 byte  (Offset 12)
    char c;     // 1 byte  (Offset 13)
    // 2 bytes padding de tong kich thuoc chia het cho 8
}; // sizeof(OptimizedStruct) = 16 bytes! Tiet kiem 33% bo nho RAM!
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Nguyên Lý Căn Lề Bộ Nhớ Phần Cứng (Hardware Bus Alignment)

Tại sao CPU không đọc dữ liệu tại bất kỳ địa chỉ nào mà đòi hỏi căn lề (*Data Alignment*)?

1. **Đường truyền Bus dữ liệu:** Bộ vi xử lý hiện đại (x86-64) đọc dữ liệu từ RAM vào CPU Cache theo từng khối 32-bit, 64-bit, hoặc Cache Line 64 bytes.
2. **Chi phí truy cập lệch lề (Misaligned Access):** Nếu một biến số thực 8 bytes (`double`) nằm vắt ngang qua ranh giới của hai khối 8 bytes, CPU buộc phải thực hiện **HAI chu kỳ đọc bộ nhớ riêng biệt**, sau đó dùng các lệnh dịch bit (`shl`, `shr`) và ghép bit (`or`) để tái tạo lại giá trị.
3. Trình biên dịch C++ tự động chèn các byte đệm (**Padding Bytes**) để tối ưu hóa tốc độ truy xuất của CPU, ưu tiên hiệu năng thực thi hơn việc tiết kiệm vài byte bộ nhớ vật lý.

2.2 Bản Chất Con Trỏ Bản Thể (this Pointer) Dưới Tầng RAM & ABI Gọi Hàm

Một đối tượng khi được tạo ra trong RAM chỉ chiếm dung lượng bằng tổng các biến thành viên dữ liệu (cộng padding và con trỏ ảo nếu có). **Các hàm thành viên không chiếm bất kỳ byte nào trong không gian nhớ của đối tượng**. Mã máy của các hàm nằm duy nhất một bản trong phân vùng `.text`.

Làm thế nào một hàm thành viên biết nó đang thao tác trên dữ liệu của đối tượng nào? Câu trả lời là **Con trỏ bản thể `this` (Self-Referential Pointer)**.

```
class RegisterFile {
private:
    int value;
public:
    void setValue(int v) {
        this->value = v; // this tro toi chinh thuc the hien tai
    }
};
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Giao Diện Nhị Phân Ứng Dụng (ABI) Và Thanh Ghi Truyền Con Trỏ this

Khi lập trình viên viết dòng lệnh gọi hàm: `reg.setValue(42);`, trình biên dịch biến đổi lời gọi này thành một hàm dạng thủ tục mức thấp:

```
RegisterFile::setValue(&reg, 42);
```

Quy ước gọi hàm (Calling Conventions):

- **Trên kiến trúc 32-bit x86 (`__thiscall`):** Con trỏ bản thể `this` được truyền trực tiếp thông qua thanh ghi chuyên dụng **`ECX`**, trong khi các tham số còn lại được đẩy vào Stack từ phải qua trái.
- **Trên kiến trúc 64-bit (System V AMD64 ABI — Linux/macOS):** Con trỏ bản thể `this` luôn được nạp vào thanh ghi tham số đầu tiên: **`RDI`**. Tham số thứ hai được nạp vào **`RSI`**.
- **Trên kiến trúc 64-bit (Microsoft x64 ABI — Windows):** Con trỏ bản thể `this` được nạp vào thanh ghi **`RCX`**. Tham số thứ hai được nạp vào **`RDX`**.

Mã máy hợp ngữ được sinh ra minh họa cơ chế này:

```
lea rdi, [rbp-16]       ; Load dia chi doi tuong reg vao RDI (this pointer)
mov esi, 42             ; Load tham so 42 vao RSI
call RegisterFile::setValue ; Nhay toi ham thanh vien trong .text
```

Bản chất pháp lập theo ISO/IEC 14882: Trong hệ thống kiểu của chuẩn C++, `this` là một **prvalue** có kiểu là `ClassName*` (hoặc `const ClassName*` trong phương thức hằng). Vì là prvalue (giá trị thuần rvalue), lập trình viên không thể gán giá trị cho `this` (`this = ptr;` là lỗi ngữ pháp).

2.3 Vòng Đời Khởi Tạo Đối Tượng (Constructors Architecture)

Hàm dựng (**Constructor**) là hàm thành viên đặc biệt chịu trách nhiệm thiết lập trạng thái ban đầu của đối tượng và thiết lập các điều kiện bất biến (*Class Invariants*). Constructor không có kiểu trả về và mang tên trùng khớp với tên lớp.

2.3.1 Member Initializer List vs Phép Gán Trong Thân Constructor

```
class PipelineBlock {
private:
    std::string blockName;
    const int blockId;      // Const field: Bat buoc phai khoi tao bang danh sach
    int& systemRef;         // Reference field: Bat buoc phai khoi tao bang danh
    sach

public:
    // PHUONG PHAP CHUAN MUC (Member Initializer List):
    PipelineBlock(const std::string& name, int id, int& ref)
        : blockName(name), blockId(id), systemRef(ref) {
        // Truong du lieu duoc khoi tao truc tiep dung Constructor tuong ung
    }
};
```

Tiên Đề / Định Nghĩa Pháp Lập: Bản Chất Của Danh Sách Khởi Tạo Thành Viên (Member Initializer List)

Khi một đối tượng được khởi tạo, tiến trình cấp phát diễn ra theo hai giai đoạn độc lập:

1. **Giai đoạn 1: Khởi tạo các trường thành viên (Initialization Phase):** Xảy ra tại danh sách khởi tạo `: var1(val1), var2(val2)`. Mỗi trường được gọi trực tiếp hàm tạo tương ứng.
2. **Giai đoạn 2: Thực thi phần thân hàm tạo (Execution Phase):** Các dòng lệnh bên trong cặp dấu ngoặc nhọn `{ }` được thực thi. Nếu tại đây có phép gán `var1 = val1;`, trường đó đã phải trải qua việc khởi tạo mặc định từ trước, sau đó bị ghi đè bởi toán tử gán, gây lãng phí ít nhất 50% hiệu năng xử lý.

Hành Vi Không Xác Định (Undefined Behavior): Bẫy Thứ Tự Khởi Tạo Thành Viên (Initialization Order Trap)

Xét đoạn mã nguồn minh họa khiếm khuyết kinh điển:

```
class MemoryController {
private:
    int* buffer;       // KHAI BAO TRUOC -> KHOI TAO TRUOC!
    size_t bufferSize; // KHAI BAO SAU   -> KHOI TAO SAU!
```

```
public:
    // Canh bao: Lap trinh vien ghi bufferSize(size) truoc trong danh sach:
    MemoryController(size_t size)
        : bufferSize(size), buffer(new int[bufferSize]) {}
};
```

Quy tắc chuẩn quốc tế ISO/IEC 14882: Thứ tự khởi tạo các trường thành viên của một lớp **HOÀN TOÀN DỰA VÀO THỨ TỰ KHAI BÁO TRONG ĐỊNH NGHĨA LỚP**, hoàn toàn bất chấp thứ tự được liệt kê trong Member Initializer List!
Trong ví dụ trên, biến `buffer` được khai báo **TRƯỚC** biến `bufferSize`. Do đó, `buffer` sẽ được khởi tạo trước! Tại thời điểm `new int[bufferSize]` thực thi, biến `bufferSize` chưa hề được gán giá trị và đang chứa giá trị rác ngẫu nhiên của bộ nhớ. Kết quả là hệ thống cố gắng cấp phát một lượng RAM bất định, dẫn đến lỗi sập tiến trình (**Segmentation Fault**)!

2.3.2 Hàm Tạo Ủy Quyền & Khởi Tạo Thành Viên Mặc Định (C++11)

Chuẩn C++11 bổ sung hai cơ chế giảm thiểu sự trùng lặp mã nguồn trong khởi tạo:

1. **Khởi tạo thành viên tại chỗ (Default Member Initializers):** Cho phép gán giá trị mặc định trực tiếp tại dòng khai báo biến:

```
    class ServerConfig {
        int timeoutMs = 5000;
        bool keepAlive = true;
    };

```

2. **Hàm tạo ủy quyền (Delegating Constructors):** Cho phép một Constructor gọi một Constructor khác của cùng lớp thông qua danh sách khởi tạo:

```
    class Vector3D {
    public:
        double x, y, z;
        Vector3D(double a, double b, double c) : x(a), y(b), z(c) {}
        Vector3D() : Vector3D(0.0, 0.0, 0.0) {} // Uy quyen cho Constructor
    3 tham so
    };

```

2.3.3 Ngăn Chặn Ép Kiểu Ngầm Định Với Từ Khóa `explicit`

Trong C++, bất kỳ Constructor nào có thể được gọi với **MỘT tham số duy nhất** đều mặc định đóng vai trò là một hàm chuyển đổi kiểu ngầm định (*Converting Constructor*):

```
class BufferPool {
public:
    // explicit ngan chan compiler tu dong ep kieu int thanh BufferPool
    explicit BufferPool(int capacity) { /* ... */ }
};
```

```
6
7  void allocate(const BufferPool& pool);
8
9  void test() {
10     // BufferPool p = 100; // LOI BIEN DICH: explicit chan ep kieu ngam dinh
11     BufferPool p(100);       // HOP LE: Khoi tao tuong minh
12     allocate(BufferPool(50)); // HOP LE
13 }
```

2.4 Hàm Hủy (Destructor) & Triết Lý Thiết Kế RAII

Hàm hủy (**Destructor**) có cú pháp `~ClassName()`, không nhận tham số và không trả về giá trị. Destructor được hệ thống tự động kích hoạt khi đối tượng kết thúc vòng đời.

Tiên Đề / Định Nghĩa Pháp Lập: Triết Lý Thiết Kế RAII (Resource Acquisition Is Initialization)

Được phát minh bởi Bjarne Stroustrup, **RAII** là mô thức thiết kế phần mềm quan trọng nhất của C++ hiện đại. Tư tưởng cốt lõi của RAII thiết lập định đề:

"Mọi tài nguyên hệ thống (Bộ nhớ RAM, File Descriptor, Socket Mạng, Mutex Lock) phải được gắn chặt vào Vòng đời của một Đối tượng Ngăn xếp (Stack-allocated Object)."

- **Cấp phát tài nguyên** được thực hiện trong Constructor.
- **Giải phóng tài nguyên** được ủy thác hoàn toàn tự động cho Destructor.

Nhờ cơ chế **Trải cuộn Ngăn xếp (Stack Unwinding)**, cho dù hàm kết thúc bình thường qua lệnh `return` hay bị gián đoạn đột ngột bởi một ngoại lệ (`throw`), toàn bộ tài nguyên đều được dọn dẹp sạch sẽ 100%, loại trừ hoàn toàn nguy cơ rò rỉ tài nguyên.

2.5 Thành Viên Static, Const và Kỹ Thuật Hàm Bạn (Friend)

2.5.1 Thành Viên Tĩnh (Static Data & Methods)

- **Biến thành viên tĩnh (`static`):** Không thuộc về bất kỳ đối tượng cụ thể nào của lớp. Nó chỉ tồn tại **DUY NHẤT MỘT BẢN SAO** trong toàn bộ tiến trình, được lưu trữ tại phân vùng `.data` hoặc `.bss`. Kích thước của đối tượng hoàn toàn không chứa biến `static`.
- **Phương thức tĩnh (`static` methods):** Không nhận con trỏ bản thể `this`. Do đó, phương thức tĩnh **không thể truy cập các biến thành viên phi tĩnh** và không thể gọi các phương thức phi tĩnh nếu không thông qua một đối tượng tường minh.

2.5.2 Từ Khóa `mutable` — Ngoại Lệ Trong Phương Thức Hằng

Một phương thức được đánh dấu `const` cam kết không làm thay đổi trạng thái quan sát được (*Bitwise Constness*) của đối tượng. Tuy nhiên, trong một số kịch bản kỹ thuật (đồng bộ hóa luồng qua `std::mutex`, lưu bộ nhớ đệm cache, đếm tần suất truy cập), ta cần biến đổi một số trường dữ liệu mà không làm thay đổi ngữ nghĩa logic của đối tượng. Từ khóa **`mutable`** cho phép một trường dữ liệu được phép sửa đổi ngay bên trong phương thức `const`:

```
class DataCache {
private:
    std::string cachedValue;
    mutable size_t accessCounter = 0; // Duoc phep bien doi trong const method

public:
    std::string readData() const {
        ++accessCounter; // Hop le nho tu khoa mutable
        return cachedValue;
    }
};
```

2.5.3 Quan Hệ Bạn Bè (Friendship Architecture)

Từ khóa `friend` cho phép một hàm tự do hoặc một lớp khác có quyền truy cập vào các trường `private` và `protected` của lớp định nghĩa.

Tiên Đề / Định Nghĩa Pháp Lập: Ba Định Đề Bất Biến Về Quan Hệ Bạn Bè (Friendship Postulates)

1. **Tính Không Đối Xứng (Not Symmetric):** Nếu lớp A khai báo lớp B là bạn, thì B có quyền xem trường ẩn của A, nhưng A hoàn toàn không có quyền xem trường ẩn của B.
2. **Tính Không Bắc Cầu (Not Transitive):** Nếu A là bạn của B, và B là bạn của C, thì A hoàn toàn không mặc nhiên là bạn của C.
3. **Tính Không Kế Thừa (Not Inherited):** Lớp con của một lớp bạn không được thừa hưởng quyền bạn bè từ lớp cha.

CHƯƠNG 3

Quản trị Bộ nhớ, Move Semantics & Nạp chồng Toán tử (Memory & Overloading)

Tiên Đề / Định Nghĩa Pháp Lập: Nguyên Lý Quản Trị Tài Nguyên Bộ Nhớ

Khả năng kiểm soát bộ nhớ RAM ở cấp độ byte là phẩm chất định nghĩa nên uy lực của C++. Tuy nhiên, tự do đi liền với trách nhiệm: Một kiến trúc phần mềm chuẩn mực phải đảm bảo hai điều kiện biên bất biến:

1. **Tính Toàn Vẹn Sở Hữu (Ownership Integrity):** Mỗi khối tài nguyên động trên Heap phải có một chủ sở hữu duy nhất hoặc một nhóm chủ sở hữu được kiểm soát nghiêm ngặt.
2. **Tính Bất Biến Về Vòng Đời (Lifecycle Invariant):** Tài nguyên phải được giải phóng đúng một lần duy nhất (*Exactly-once Destruction*) ngay khi chủ sở hữu cuối cùng kết thúc vòng đời.

3.1 Sao Chép Nông (Shallow Copy) vs Sao Chép Sâu (Deep Copy)

Khi lập trình viên không khai báo Copy Constructor hoặc Copy Assignment Operator, trình biên dịch sẽ tự động sinh ra một phiên bản mặc định thực hiện **Sao chép nông (Shallow Copy / Memberwise Copy)**.

3.1.1 Cơ Chế Phá Hủy Bộ Nhớ Của Shallow Copy (Double-Free Hazard)

Xét một lớp quản lý một mảng động trên Heap:

```
1 class RawArray {
2 public:
```

```
    int* data;
    size_t size;

    RawArray(size_t s) : size(s), data(new int[s]) {}
    ~RawArray() { delete[] data; }
    // No custom Copy Constructor -> Compiler generates shallow copy!
};

void defectiveExecution() {
    RawArray a(100);
    RawArray b = a; // SHALLOW COPY: b.data gets the exact pointer value of a.data
    !
} // Scope exits: Double-free error occurs here!
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Giải Phẫu Hiện Tượng Double-Free Dưới Tầng Bộ Nhớ

Hãy khảo sát trạng thái của bộ nhớ trong hàm `defectiveExecution()`:

1. Khi `b = a` được thực thi, trình biên dịch chỉ đơn thuần sao chép giá trị số nguyên của con trỏ: `b.data = a.data`.
2. **Hiện tượng Con trỏ trùng lặp (Pointer Aliasing):** Cả `a.data` và `b.data` đều cùng trỏ vào **DUY NHẤT MỘT ĐỊA CHỈ HEAP** (ví dụ `0x00A01234`).
3. **Giai đoạn Hủy (Destruction):**
   - Theo quy tắc LIFO của Stack, đối tượng `b` bị hủy trước. Destructor `~RawArray()` của `b` thực thi lệnh `delete[] b.data`, giải phóng khối bộ nhớ tại `0x00A01234` về cho hệ điều hành. Vùng nhớ này bị đánh dấu là *Free Chunk*.
   - Ngay sau đó, đối tượng `a` bị hủy. Destructor của `a` tiếp tục gọi `delete[] a.data` trên cùng địa chỉ `0x00A01234`!
4. **Hậu quả hệ thống:** Việc giải phóng một vùng nhớ đã được giải phóng từ trước gọi là **Lỗi Double-Free**. C/C++ Memory Allocator phát hiện sự sai lệch trong danh sách liên kết của Heap và ném tín hiệu **`SIGABRT`**, đình chỉ tiến trình ngay tức khắc.

3.1.2 Giải Pháp Chuẩn Mực: Sao Chép Sâu (Deep Copy)

Sao chép sâu đòi hỏi việc cấp phát một khối bộ nhớ mới hoàn toàn độc lập trên Heap, sau đó sao chép toàn bộ nội dung dữ liệu từ nguồn sang đích:

```
// Deep Copy Constructor
RawArray::RawArray(const RawArray& other)
    : size(other.size), data(new int[other.size]) {
    for (size_t i = 0; i < size; ++i) {
        data[i] = other.data[i];
    }
}
```

3.2 Hệ Thống Phân Loại Giá Trị & Ngữ Nghĩa Di Dời (Move Semantics)

3.2.1 Phân Loại Giá Trị Chuẩn Quốc Tế ISO/IEC 14882 (Value Categories)

Trong chuẩn C++11, để hỗ trợ việc chuyển giao quyền sở hữu tài nguyên mà không cần sao chép, ủy ban ISO đã thiết lập cây phân loại giá trị (*Value Categories*) dựa trên hai tiêu chí độc lập:

1. **Có Định Danh Bộ Nhớ (Has Identity):** Chương trình có thể xác định được địa chỉ bộ nhớ cụ thể (`&expr`) của đối tượng.
2. **Có Thể Di Dời Tài Nguyên (Can be Moved):** Trạng thái nội tại của đối tượng có thể được chuyển nhượng cho đối tượng khác một cách an toàn mà không phá vỡ tính bất biến của chương trình.

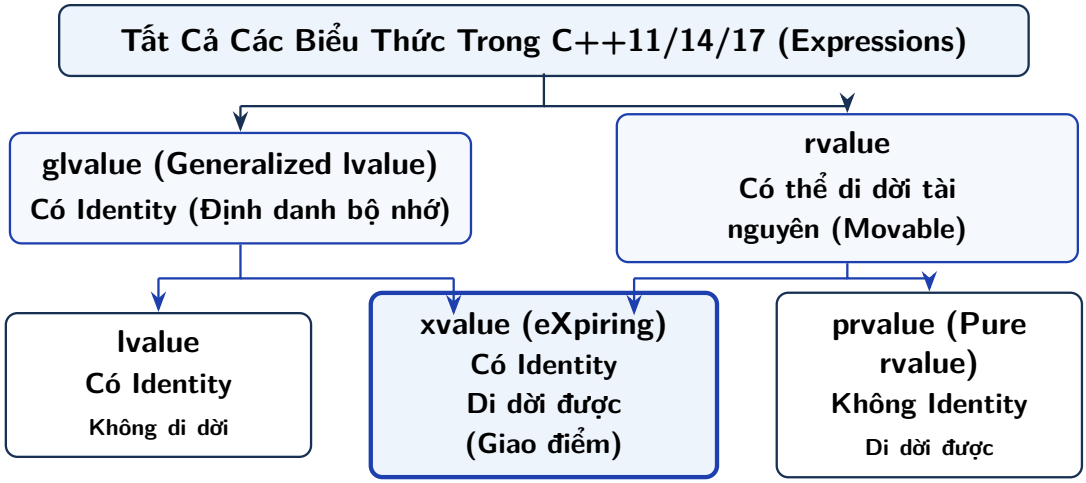


Figure 3.1: Cây phân loại giá trị chuẩn mực quốc tế ISO/IEC 14882 (Value Categories).

- **lvalue (Locator Value):** Đại diện cho một đối tượng có vị trí bộ nhớ xác định và có tên (ví dụ: tên biến, hàm trả về tham chiếu lvalue `T&`).
- **prvalue (Pure Rvalue):** Đại diện cho các giá trị tạm thời tính toán tức thời hoặc hằng số nguyên thủy (ví dụ: số `42`, kết quả biểu thức `a + b`, hàm trả về theo giá trị `T`). Từ C++17, prvalue được định nghĩa lại là *công thức khởi tạo đối tượng*, kích hoạt cơ chế **Triệt tiêu Sao chép Bảo đảm (Guaranteed Copy Elision)**.
- **xvalue (eXpiring Value):** Đại diện cho một đối tượng có danh tính bộ nhớ nhưng vòng đời sắp kết thúc, cho phép chuyển giao tài nguyên (ví dụ: kết quả của `std::move(var)`).

3.2.2 Tham Chiếu Rvalue (`T&&`) & Ngữ Nghĩa Di Dời (Move Semantics)

Trong C++98, việc truyền các đối tượng lớn (chuỗi ký tự, mảng động, ma trận) ra khỏi hàm luôn đòi hỏi một phép sao chép sâu vô cùng đắt đỏ. Chuẩn C++11 giải quyết triệt để vấn đề này thông qua **Tham chiếu Rvalue (`T&&`)**:

```cpp
#include <iostream>
#include <utility>

class HeavyBuffer {
public:
    size_t size;
    int* data;

    // 1. Constructor
    HeavyBuffer(size_t s) : size(s), data(new int[s]) {}

    // 2. Destructor
    ~HeavyBuffer() { delete[] data; }

    // 3. Copy Constructor (Deep Copy - O(N))
    HeavyBuffer(const HeavyBuffer& other) : size(other.size), data(new int[other.size]) {
        for (size_t i = 0; i < size; ++i) data[i] = other.data[i];
    }

    // 4. Move Constructor (Resource Transfer - O(1))
    HeavyBuffer(HeavyBuffer&& other) noexcept : size(other.size), data(other.data) {
        // Chuyen giao con tro va vo hieu hoa doi tuong nguon
        other.data = nullptr;
        other.size = 0;
    }
};
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Bản Chất Của `std::move` Dưới Tầng Trình Biên Dịch

Rất nhiều kỹ sư ngộ nhận rằng hàm `std::move` thực hiện thao tác di dời dữ liệu trong bộ nhớ. **Đây là một hiểu lầm tai hại!**
Dưới góc nhìn mã máy, `std::move` hoàn toàn **KHÔNG SINH RA BẤT KỲ DÒNG MÃ THỰC THI NÀO**! Về bản chất, `std::move` chỉ là một phép **Ép Kiểu Tĩnh Không Điều Kiện (Unconditional Static Cast)**:

```
static_cast<typename std::remove_reference<T>::type&&>(t)
```

Mục đích duy nhất của `std::move` là biến đổi một **lvalue** thành một **xvalue**, thông báo cho cơ chế giải quyết nạp chồng (*Overload Resolution*) của trình biên dịch ưu tiên lựa chọn Move Constructor hoặc Move Assignment Operator thay vì Copy Constructor.

3.3 Tiến Hóa Bộ Quy Tắc Quản Trị Tài Nguyên: Rule of 3, 5, 0

3.3.1 Quy Tắc Bộ Ba Cổ Điển (The Rule of Three — C++98)

Tiên Đề / Định Nghĩa Pháp Lập: Quy Tắc Bộ Ba (The Rule of Three)

Nếu một lớp đòi hỏi phải hiện thực hóa tường minh một trong ba phương thức sau:

1. **Hàm Hủy (Destructor)**
2. **Hàm Tạo Sao Chép (Copy Constructor)**
3. **Toán Tử Gán Sao Chép (Copy Assignment Operator)**

thì lớp đó **HẦU NHƯ CHẮC CHẮN PHẢI HIỆN THỰC HÓA CẢ BA PHƯƠNG THỨC**. Sự hiện diện của một hàm hủy tường minh chứng minh rằng lớp đang nắm giữ một tài nguyên thô; do đó các phương thức sao chép mặc định sẽ gây ra lỗi Shallow Copy.

3.3.2 Quy Tắc Bộ Năm Hiện Đại (The Rule of Five — C++11)

Sự xuất hiện của Ngữ nghĩa Di dời trong chuẩn C++11 mở rộng Rule of Three thành **Rule of Five**. Một lớp quản lý tài nguyên phải thiết lập 5 phương thức đặc biệt:

1. **Destructor:** `~Resource()`
2. **Copy Constructor:** `Resource(const Resource&)`
3. **Copy Assignment Operator:** `Resource& operator=(const Resource&)`
4. **Move Constructor:** `Resource(Resource&&) noexcept`
5. **Move Assignment Operator:** `Resource& operator=(Resource&&) noexcept`

Cảnh Báo Kỹ Thuật & Rủi Ro Biên Dịch: Quy Tắc Vô Hiệu Hóa Hàm Di Dời Ngầm Định (Move Generation Inhibition)

Theo chuẩn ISO C++: Nếu lập trình viên khai báo bất kỳ hàm nào trong số **Destructor, Copy Constructor, hoặc Copy Assignment Operator**, trình biên dịch sẽ **TỰ ĐỘNG KHÓA VÀ KHÔNG SINH RA Move Constructor và Move Assignment Operator ngầm định**!
Hậu quả là các lời gọi `std::move(obj)` sẽ **âm thầm thoái hóa thành phép sao chép sâu** (*Silent Fallback to Deep Copy*) mà không hề có bất kỳ cảnh báo biên dịch nào! Để khôi phục, lập trình viên bắt buộc phải yêu cầu trình biên dịch sinh tường minh:

```
Resource(Resource&&) noexcept = default;
Resource& operator=(Resource&&) noexcept = default;
```

3.3.3 Thành Ngữ Sao Chép và Hoán Vị (The Copy-and-Swap Idiom)

Để hiện thực hóa cả Copy Assignment và Move Assignment một cách thanh lịch, an toàn ngoại lệ tuyệt đối mà không lặp lại mã nguồn, kỹ sư C++ sử dụng kỹ thuật **Copy-and-Swap**:

```cpp
#include <utility>
#include <algorithm>

class ManagedBuffer {
private:
    size_t length;
    char* buffer;

public:
    // 1. Constructor
    ManagedBuffer(size_t len = 0)
        : length(len), buffer(len ? new char[len] : nullptr) {}

    // 2. Destructor
    ~ManagedBuffer() { delete[] buffer; }

    // 3. Copy Constructor (Deep Copy - Protected against nullptr UB)
    ManagedBuffer(const ManagedBuffer& other)
        : length(other.length), buffer(other.length ? new char[other.length] :
    nullptr) {
        if (length > 0) {
            std::copy(other.buffer, other.buffer + length, buffer);
        }
    }

    // 4. Move Constructor (Zero Overhead Transfer)
    ManagedBuffer(ManagedBuffer&& other) noexcept
        : ManagedBuffer() { // Delegate to Default Ctor
        swap(*this, other);
    }

    // Friend swap function: No-fail guarantee
    friend void swap(ManagedBuffer& first, ManagedBuffer& second) noexcept {
        using std::swap;
        swap(first.length, second.length);
        swap(first.buffer, second.buffer);
    }

    // 5. UNIFIED ASSIGNMENT OPERATOR (Covers both Copy and Move!)
    // Parameter passed BY VALUE triggers Copy Ctor for lvalue or Move Ctor for
    rvalue!
    ManagedBuffer& operator=(ManagedBuffer other) noexcept {
        swap(*this, other); // Swap internal state with temporary copy
        return *this;
    } // temporary other is destroyed here, cleanly freeing old resource
};
```

3.3.4 Quy Tắc Số Không (The Rule of Zero — Modern C++)

Tiên Đề / Định Nghĩa Pháp Lập: Đỉnh Cao Thiết Kế: Quy Tắc Số Không (The Rule of Zero)

Trong kiến trúc hướng đối tượng C++ hiện đại, lập trình viên **KHÔNG NÊN TỰ VIẾT BẤT KỲ PHƯƠNG THỨC NÀO TRONG BỘ NĂM**. Thay vì quản lý con trỏ thô, hãy ủy thác toàn bộ việc quản lý tài nguyên cho các lớp thư viện chuẩn đã đạt chuẩn RAII như `std::unique_ptr`, `std::vector`, hoặc `std::string`. Khi đó, trình biên dịch sẽ tự động sinh ra toàn bộ các hàm tạo sao chép, di chuyển và hàm hủy với độ an toàn và hiệu năng tối ưu tuyệt đối.

3.4 Nạp Chồng Toán Tử Toàn Diện (Operator Overloading Deep Dive)

3.4.1 Toán Tử Tiền Tố (++x) vs Hậu Tố (x++)

```
class Counter {
private:
    int value = 0;
public:
    // 1. PREFIX (++c): Returns reference to current instance
    Counter& operator++() {
        ++value;
        return *this; // Optimal: No temporary object created
    }

    // 2. POSTFIX (c++): Takes dummy int parameter to differentiate
    Counter operator++(int) {
        Counter temp = *this; // Mandatory: Must create a temporary copy
        ++value;              // Increment actual state
        return temp;          // Return old value before increment
    } // Overhead: Triggers Copy Constructor
};
```

Hệ luận kỹ thuật: Trong các vòng lặp thuật toán, luôn ưu tiên sử dụng toán tử tiền tố (`++it`) thay vì hậu tố (`it++`) để triệt tiêu chi phí khởi tạo đối tượng tạm thời.

3.4.2 Nạp Chồng new/delete Cấp Lớp (Class-specific Allocators) & Placement New

Placement New cho phép lập trình viên khởi tạo một đối tượng trực tiếp trên một địa chỉ bộ nhớ đã được phân bổ sẵn mà **hoàn toàn không thực hiện bất kỳ lệnh xin cấp phát RAM nào**:

```
#include <new> // Required for placement new

alignas(SocketConnection) char arenaBuffer[sizeof(SocketConnection)];

```

```cpp
void testPlacementNew() {
    // 1. Construct object directly on arenaBuffer:
    SocketConnection* pConn = new (arenaBuffer) SocketConnection(9090);

    // 2. DESTRUCTION RULE: STRICTLY FORBIDDEN TO CALL delete pConn!
    // Since arenaBuffer is on Stack/Static, calling delete triggers a crash!

    // 3. MUST MANUALLY INVOKE DESTRUCTOR EXPLICITLY:
    pConn->~SocketConnection();
}
```

3.5 Kiến Trúc Con Trỏ Thông Minh Chuẩn C++11 (Smart Pointers Architecture)

Để chấm dứt kỷ nguyên rò rỉ bộ nhớ của con trỏ thô (*Raw Pointers*), chuẩn C++11 giới thiệu bộ ba con trỏ thông minh trong thư viện `<memory>`.

3.5.1 `std::unique_ptr<T>` — Quyền Sở Hữu Độc Quyền (Exclusive Ownership)

- **Đặc tính kiến trúc:** Quản lý một đối tượng duy nhất. Không thể sao chép (`Copy Constructor = delete`), chỉ có thể di dời quyền sở hữu thông qua Move Semantics (`std::move`).
- **Hiệu năng và Tối ưu hóa Lớp Cơ sở Rỗng (EBO):** Khi sử dụng bộ giải phóng mặc định (`std::default_delete<T>`), trình biên dịch áp dụng EBO để kích thước của `std::unique_ptr<T>` đúng bằng **MỘT con trỏ thô** (8 bytes trên 64-bit). Tuy nhiên, nếu truyền vào một con trỏ hàm giải phóng tùy biến (`void(*)(T*)`), kích thước của nó sẽ tăng gấp đôi lên **16 bytes**.

3.5.2 `std::shared_ptr<T>` — Quyền Sở Hữu Chia Sẻ (Shared Ownership)

Nhiều con trỏ `shared_ptr` có thể cùng trỏ tới một đối tượng. Đối tượng sẽ được tự động giải phóng khi con trỏ `shared_ptr` cuối cùng bị hủy.

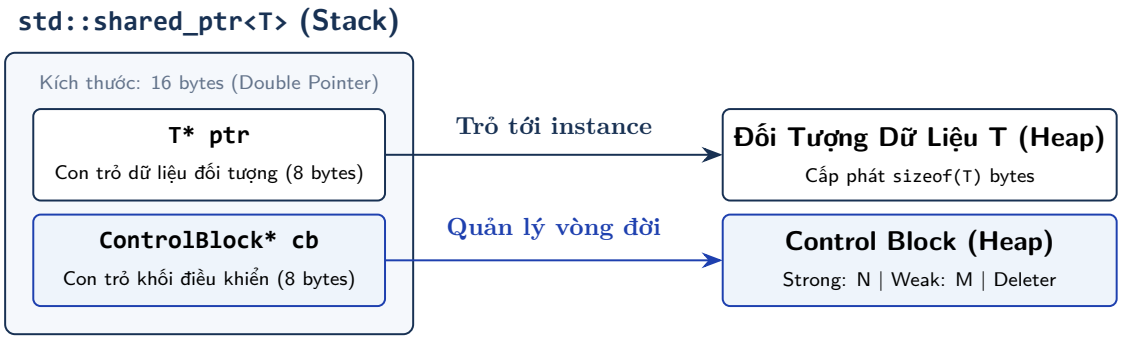


Figure 3.2: Kiến trúc bộ nhớ hai con trỏ và Khối Điều Khiển (Control Block) của `std::shared_ptr`.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Kiến Trúc Khối Điều Khiển (Control Block) & Cạm Bẫy Ghim Bộ Nhớ Của `std::make_shared`

Kích thước của một `std::shared_ptr` gấp đôi con trỏ thô (16 bytes trên nền tảng 64-bit), bao gồm một con trỏ trỏ tới dữ liệu và một con trỏ trỏ tới Khối Điều Khiển (*Control Block*).

So sánh giữa `std::make_shared` và cấp phát hai lần:

1. **Lợi thế của `std::make_shared<T>()`:** Cấp phát duy nhất một khối RAM liên tục chứa cả Control Block và Đối tượng dữ liệu. Tối ưu hóa tính cục bộ bộ nhớ đệm (*Cache Locality*) và loại trừ chi phí gọi allocator hai lần.
2. **Cạm bẫy ghim bộ nhớ (Memory Retention Trap):** Vì đối tượng `T` và Control Block nằm chung trên một khối nhớ Heap duy nhất:
   - Khi Strong Reference Count giảm về 0, Destructor của `T` được thực thi ngay lập tức.
   - Tuy nhiên, **TOÀN BỘ KHỐI BỘ NHỚ HEAP KHÔNG THỂ ĐƯỢC GIẢI PHÓNG** chừng nào Weak Reference Count vẫn còn lớn hơn 0!
   - Nếu đối tượng `T` có kích thước lớn (ví dụ ma trận 50 MB) và có một đối tượng quan sát nắm giữ `std::weak_ptr` trong thời gian dài, toàn bộ 50 MB RAM sẽ bị ghim chặt trong bộ nhớ! Trong kịch bản này, sử dụng `std::shared_ptr<T>(new T())` là lựa chọn an toàn hơn vì vùng nhớ 50 MB của `T` được giải phóng ngay khi strong ref về 0.

3.5.3 `std::weak_ptr<T>` — Phá Vỡ Vòng Lặp Tham Chiếu (Circular Reference)

Nếu hai đối tượng `A` và `B` cùng sở hữu `std::shared_ptr` trỏ chéo lẫn nhau, bộ đếm Strong Reference Count của cả hai không bao giờ giảm về 0, dẫn tới rò rỉ bộ nhớ vĩnh viễn.

Giải pháp: Thay thế một trong hai liên kết bằng **`std::weak_ptr<T>`**. `weak_ptr` không làm tăng Strong Reference Count. Khi muốn sử dụng dữ liệu, nó thăng hạng an toàn thành `shared_ptr` thông qua phương thức `wp.lock()`; nếu đối tượng đã bị giải phóng, `lock()` trả về con trỏ rỗng an toàn.

CHƯƠNG 4

Kế thừa, Bố cục Bộ nhớ Nhị phân & Mô hình hóa UML (Inheritance)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Tính Kế Thừa (Inheritance)

Kế thừa (Inheritance) là cơ chế cho phép một lớp mới (**Lớp dẫn xuất / Lớp con** — **Derived Class**) tiếp nhận lại toàn bộ cấu trúc dữ liệu và hành vi của một lớp đã tồn tại (**Lớp cơ sở / Lớp cha** — **Base Class**), đồng thời mở rộng hoặc tái định nghĩa các hành vi chuyên biệt mà không cần viết lại mã nguồn ban đầu.

4.1 Bản Chất Quan Hệ Hướng Đối Tượng & Ký Hiệu UML

Trong thiết kế phần mềm, việc phân định rạch ròi các mối quan hệ giữa các lớp là nền tảng sống còn của kiến trúc:

1. **Mối quan hệ IS-A (Kế thừa / Khái quát hóa — Generalization):**
   - Thể hiện tính phân loại bản thể luận: *"Hình tròn LÀ MỘT Hình học"* (`Circle IS-A Shape`).
   - Được biểu diễn bằng **Kế thừa Công khai (Public Inheritance)**. Đối tượng lớp con phải thay thế được hoàn toàn cho đối tượng lớp cha trong mọi hoàn cảnh mà không phá vỡ tính đúng đắn của chương trình (Nguyên lý Thay thế Liskov — Liskov Substitution Principle).
   - *Ký hiệu UML chuẩn:* Mũi tên tam giác rỗng hướng từ lớp con lên lớp cha ($\triangle-$).
2. **Mối quan hệ HAS-A (Sở hữu / Hợp thành — Composition vs Aggregation):**

- **Hợp thành (Composition):** Quan hệ sở hữu vòng đời khăng khít. Thành phần con không thể tồn tại độc lập nếu đối tượng chứa nó bị hủy (ví dụ: *"Ô tô CÓ Động cơ"*). *Ký hiệu UML:* Hình thoi đặc (♦−).
- **Tụ hợp (Aggregation):** Quan hệ sở hữu lỏng lẻo. Thành phần con có vòng đời độc lập với đối tượng chứa nó (ví dụ: *"Trường học CÓ Giáo viên"*). *Ký hiệu UML:* Hình thoi rỗng (◇−).

3. **Mối quan hệ USES-A (Phụ thuộc — Dependency):** Một lớp sử dụng dịch vụ của một lớp khác thông qua tham số hàm mà không nắm giữ biến thành viên lâu dài. *Ký hiệu UML:* Mũi tên nét đứt (- - →).

4.1.1 Bố Cục Bộ Nhớ Đối Tượng Kế Thừa Đơn (Single Inheritance Memory Layout)

Trong C++, lớp con kế thừa từ lớp cha được tổ chức theo nguyên lý lồng ghép cấu trúc nhị phân liên tục trong RAM:

Không Gian Bộ Nhớ Nhị Phân Của Đối Tượng Phái Sinh (Derived Object)

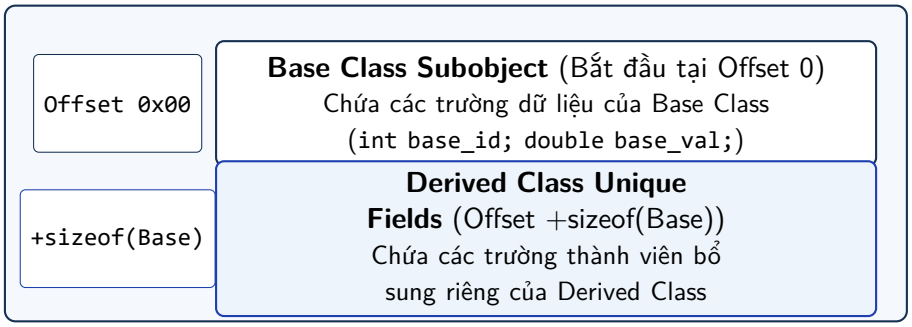


Figure 4.1: Bố cục bộ nhớ nhị phân của đối tượng phái sinh trong Kế thừa Đơn.

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Đặc Điểm Con Trỏ Trong Kế Thừa Đơn

Vì Subobject của Base nằm tại vị trí bắt đầu (Offset 0) của đối tượng Derived, nên:

```
&derived == static_cast<Base*>(&derived)
```

Địa chỉ con trỏ của đối tượng con và địa chỉ con trỏ khi ép kiểu lên đối tượng cha là **hoàn toàn trùng khớp về mặt giá trị số nguyên**! Trình biên dịch không cần sinh thêm bất kỳ lệnh cộng trừ offset nào khi thực hiện phép Upcast trong kế thừa đơn.

4.2 Ma Trận Biến Đổi Quyền Truy Cập Trong Kế Thừa

Quyền truy cập của các thành viên kế thừa từ lớp cha vào lớp con phụ thuộc vào kiểu kế thừa:

Table 4.1: Ma trận biến đổi quyền truy cập các trường dữ liệu trong kế thừa

Quyền Tại Lớp Base	Kế Thừa Public	Kế Thừa Protected	Kế Thừa Private
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	*Bị ẩn hoàn toàn*	*Bị ẩn hoàn toàn*	*Bị ẩn hoàn toàn*

- **Kế thừa Public:** Thiết lập mối quan hệ IS-A thực sự. Cho phép chuyển đổi con trỏ tự do từ con lên cha ở bất kỳ đâu.
- **Kế thừa Private:** Thiết lập mối quan hệ *"Được cài đặt bằng cách sử dụng"* (*Implemented-in-terms-of*). Toàn bộ giao diện của cha bị đóng kín bên trong con.
- **Tối ưu hóa Lớp Cơ sở Rỗng (Empty Base Optimization — EBO):** Theo chuẩn C++, một lớp hoàn toàn rỗng không chứa biến thành viên vẫn phải có kích thước tối thiểu sizeof = 1 byte để bảo đảm mỗi đối tượng có một địa chỉ phân biệt. Tuy nhiên, khi kế thừa theo dạng `private`, trình biên dịch C++ áp dụng EBO để triệt tiêu byte này: Kích thước của lớp con **không bị tăng thêm 1 byte nào**. Kỹ thuật này được STL sử dụng triệt để trong các bộ nén con trỏ và bộ cấp phát tĩnh (`std::allocator`).

4.3 Vòng Đời Khởi Tạo và Hủy Kế Thừa

Tiên Đề / Định Nghĩa Pháp Lập: Định Đề Bất Biến Về Trình Tự Vòng Đời Kế Thừa

Khi khởi tạo một đối tượng lớp con:

1. **Khởi tạo Lớp Cơ sở (Base First):** Constructor của lớp cha được thực thi đầu tiên để bảo đảm phần nền tảng được định hình vững chắc.
2. **Khởi tạo Các Thành Viên Lớp Con (Member Fields):** Các biến dữ liệu riêng của lớp con được khởi tạo theo thứ tự khai báo.
3. **Thực thi Thân Constructor Lớp Con (Derived Body):** Phần thân hàm tạo của lớp con hoàn thiện các nghiệp vụ cuối cùng.

Khi đối tượng bị tiêu hủy:

Trình tự hủy diễn ra theo đúng quy tắc LIFO (Last-In, First-Out) ngược lại hoàn toàn:

Destructor Lớp Con chạy trước $\longrightarrow$ Dọn dẹp thành viên $\longrightarrow$ Destructor Lớp Cha chạy sau cùng.

Cảnh Báo Kỹ Thuật & Rủi Ro Biên Dịch: Yêu Cầu Gọi Base Copy Constructor Khi Tự Viết Copy Ctor Ở Lớp Con

Khi lập trình viên tự tay viết Copy Constructor cho lớp con:

```
1 class Base {
```

```
public:
    int baseVal;
    Base(int v) : baseVal(v) {}
    Base(const Base& other) : baseVal(other.baseVal) {}
};

class Derived : public Base {
public:
    int derivedVal;
    Derived(int b, int d) : Base(b), derivedVal(d) {}

    // DANGEROUS OMISSION: Quen goi Base(other) trong member initializer list!
    Derived(const Derived& other) : derivedVal(other.derivedVal) {
        // Compiler se am tham goi Default Constructor Base::Base()!
        // Neu Base khong co Default Ctor -> Loi bien dich.
        // Neu Base co Default Ctor -> baseVal bi dat lai mac dinh, mat du lieu sao chep!
    }

    // HIEN THUC HOA CHUAN MUC:
    // Derived(const Derived& other) : Base(other), derivedVal(other.derivedVal) {}
};
```

4.4 Hiện Tượng Che Khuất Tên (Name Hiding) vs Nạp Chồng

Trong C++, cơ chế phân giải tên (*Name Lookup*) tuân theo nguyên tắc: **Dừng lại ở phạm vi gần nhất tìm thấy định danh trùng khớp**.

```
class Base {
public:
    void process(int x) { std::cout << "Base::process(int)\n"; }
    void process(double x) { std::cout << "Base::process(double)\n"; }
};

class Derived : public Base {
public:
    // Che khuat toan bo cac ham Base::process:
    void process(std::string s) { std::cout << "Derived::process(string)\n"; }
};

void test() {
    Derived d;
    d.process("Hello"); // Hop le: Goi Derived::process(string)
    // d.process(10);   // LOI BIEN DICH: Compiler khong the tim thay process(int)!
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Bản Chất Của Hiện Tượng Name Hiding

Tại sao `d.process(10)` không gọi `Base::process(int)`?

1. Khi phân tích cú pháp biểu thức `d.process(10)`, trình biên dịch bắt đầu tìm kiếm tên định danh `process` từ phạm vi của lớp `Derived`.
2. Nó tìm thấy `void Derived::process(std::string)`.
3. **Cơ chế dừng tìm kiếm:** Khi đã tìm thấy ít nhất một tên khớp trong một phạm vi, trình biên dịch **DỪNG TÌM KIẾM NGAY LẬP TỨC** và không bao giờ quét tiếp lên phạm vi của lớp cha `Base`!
4. Sau đó, nó tiến hành phân giải nạp chồng trên tập hợp các hàm tìm được tại lớp con. Vì tập hợp này chỉ có phiên bản nhận `std::string`, việc truyền số nguyên `10` bị báo lỗi biên dịch.
5. **Giải pháp chuẩn mực:** Sử dụng từ khóa **`using`** để kéo toàn bộ họ hàm của cha vào lớp con:

```
    class Derived : public Base {
    public:
        using Base::process; // Keo toan bo cac ban nap chong cua Base
     vao Derived
        void process(std::string s);
    };

```

4.5 Cắt Xén Đối Tượng Đa Hình (Polymorphic Object Slicing)

Hành Vi Không Xác Định (Undefined Behavior): Hiện Tượng Cắt Xén Đối Tượng Khi Truyền Tham Trị

Xét đoạn mã sau:

```
void renderEntity(Base b) { // Truyen theo GIA TRI (Pass-by-value)
    // ...
}

void execution() {
    Derived d(10, 20);
    renderEntity(d); // HIEN TUONG CAT XEN XAY RA TAI DAY
}
```

Giải phẫu cơ chế cắt xén dưới RAM:

1. Hàm `renderEntity` yêu cầu một biến có kích thước đúng bằng sizeof(Base) trên ngăn xếp Stack của nó.
2. Khi truyền đối tượng `d` vào, trình biên dịch gọi Copy Constructor của lớp cơ sở: `Base::Base(const Base&)`.
3. Trình biên dịch sao chép chính xác phần Subobject của Base từ `d`, và **GỌT BỎ HOÀN TOÀN (SLICE OFF)** toàn bộ các trường dữ liệu riêng và

bản chất đa hình của `Derived`!
4. Mọi hành vi đa hình và dữ liệu của lớp con bị tiêu biến hoàn toàn mà không có bất kỳ cảnh báo biên dịch nào.
5. **Quy tắc Vàng:** Trong hướng đối tượng, luôn luôn truyền đối tượng đa hình qua **Tham Chiếu Hằng (`const Base&`)** hoặc **Con Trỏ (`Base*`)**, tuyệt đối cấm truyền theo giá trị.

4.6 Đa Kế Thừa, Bài Toán Kim Cương & Kế Thừa Ảo

4.6.1 Mô Hình Bài Toán Kim Cương (The Diamond Problem)

Đa kế thừa (*Multiple Inheritance*) cho phép một lớp kế thừa từ nhiều lớp cha độc lập. Tuy nhiên, nó dẫn tới cấu trúc đồ thị hình thoi kinh điển:

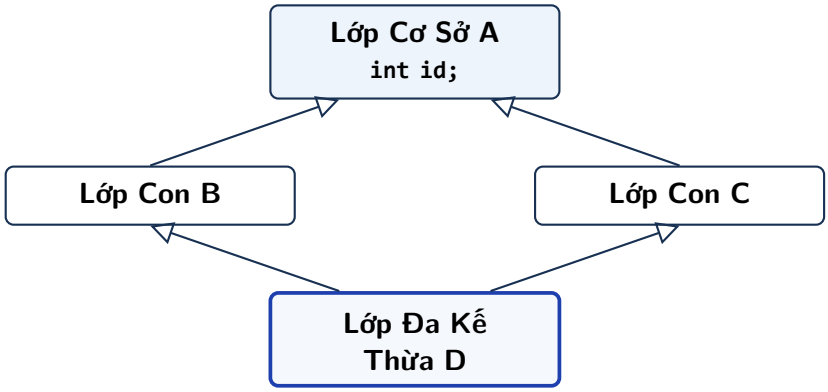


Figure 4.2: Mô hình phân cấp kế thừa hình thoi chuẩn UML (The Diamond Problem).

- **Nếu không có kế thừa ảo:** Đối tượng `D` trong RAM sẽ chứa **HAI BẢN SAO ĐỘC LẬP CỦA** A (một bản qua nhánh B, một bản qua nhánh C). Kích thước đối tượng bị nhân đôi, và lời gọi `d.id` bị trình biên dịch từ chối vì xung đột mơ hồ (*Ambiguity Error*).
- **Giải pháp: Kế Thừa Ảo (`virtual public A`):**

```
    class A { public: int id; A(int x) : id(x) {} };
    class B : virtual public A { public: B() : A(1) {} };
    class C : virtual public A { public: C() : A(2) {} };
    class D : public B, public C {
    public:
        // MOST-DERIVED CLASS EXCLUSIVE DUTY:
        // D truc tiep chiu trach nhiem khoi tao A!
        D() : A(100), B(), C() {}
    };

```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Kiến Trúc Bộ Nhớ Kế Thừa Ảo: MSVC vs Itanium ABI

Để bảo đảm trong lòng đối tượng D chỉ tồn tại duy nhất **MỘT Subobject** A **dùng chung**, hai họ trình biên dịch sử dụng hai giải pháp nhị phân độc lập:

1. **Microsoft Visual C++ (MSVC Model):** Chèn thêm con trỏ cơ sở ảo **`vbptr`** (Virtual Base Pointer) vào các lớp trung gian B và C. Con trỏ này trỏ tới bảng **Virtual Base Table (vbtbl)** chứa độ lệch động tới Subobject A dùng chung nằm ở cuối đối tượng.
2. **Itanium C++ ABI (GCC / Clang Model):** Không sử dụng con trỏ `vbptr` riêng rẽ. Thay vào đó, **Virtual Base Offsets** được lưu trữ tại các **offset âm ngay bên trong VTable** của đối tượng. Đồng thời, trình biên dịch sinh ra một bảng cấu trúc phụ gọi là **VTT (Virtual Table Table)** để truyền vào Constructor khi khởi tạo các subobject.
3. **Quy tắc Thống trị (The Dominance Rule):** Trong cây kế thừa ảo, nếu một phương thức ảo được ghi đè tại một lớp dẫn xuất B, thì định nghĩa tại B sẽ "thống trị" định nghĩa gốc tại A, cho phép trình biên dịch tự động giải quyết xung đột tên mà không báo lỗi mơ hồ.

CHƯƠNG 5

Tính Đa hình, Liên kết Động & Kiểu Trả về Đồng biến (Polymorphism)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Tính Đa Hình (Polymorphism)

Đa hình (Polymorphism) — xuất phát từ tiếng Hy Lạp cổ mang nghĩa "nhiều hình thái" — là trụ cột thứ ba và là linh hồn của Lập trình Hướng Đối tượng. Đa hình cho phép một biến con trỏ hoặc tham chiếu thuộc lớp cơ sở (`Base`) có khả năng trỏ tới đối tượng của bất kỳ lớp phái sinh nào (`Derived`), và khi thực hiện lời gọi phương thức thành viên, phiên bản hàm chính xác tương ứng với kiểu đối tượng thực tế sẽ được kích hoạt tại thời điểm thực thi.

5.1 Triết lý Đa hình & Cơ chế Liên kết (Binding)

Để hiểu sâu sắc bản chất của Đa hình, ta phân tích sự khác biệt cốt tử giữa hai cơ chế liên kết lời gọi hàm với mã máy:

1. **Liên kết Tĩnh (Early Binding / Static Binding):**
    - Xảy ra tại **thời điểm biên dịch (Compile-time)**. Trình biên dịch dựa hoàn toàn vào **kiểu dữ liệu tĩnh (Static Type)** của biến hoặc con trỏ để xác định địa chỉ hàm cần thực thi.
    - Mã máy sinh ra là một lệnh nhảy trực tiếp: `call 0x004012A0`.
    - *Ưu điểm:* Tốc độ thực thi tối đa, không tốn thêm bất kỳ byte bộ nhớ nào, cho phép trình biên dịch thực hiện tối ưu hóa nội dòng (**Function Inlining**).

- *Nhược điểm:* Thiếu sự linh hoạt, không thể thay đổi hành vi theo ngữ cảnh thời gian chạy.

2. **Liên kết Động (Late Binding / Dynamic Binding):**
   - Xảy ra tại **thời điểm thực thi (Runtime)**. Việc quyết định gọi phiên bản hàm nào được hoãn lại cho tới khi chương trình đang chạy, dựa vào **kiểu dữ liệu thực tế (Dynamic Type)** của đối tượng mà con trỏ đang trỏ tới.
   - Mã máy sinh ra là một lệnh nhảy gián tiếp thông qua con trỏ bộ nhớ: `call qword ptr [rax + offset]`.
   - *Ưu điểm:* Cho phép mở rộng hệ thống phần mềm mà không cần sửa đổi mã nguồn đã tồn tại (Nguyên lý Đóng/Mở — Open-Closed Principle).
   - *Nhược điểm:* Tốn chi phí gián tiếp (Indirection overhead), tăng kích thước đối tượng và vô hiệu hóa inlining.

Tiên Đề / Định Nghĩa Pháp Lập: Nguyên Lý Không Phụ Trội Của C++ (Zero-Overhead Principle)

Trong các ngôn ngữ như Java hay C#, mọi phương thức thành viên mặc định đều là **Dynamic Binding**. Ngược lại, triết lý thiết kế của C++ kiên định với tôn chỉ **Zero-Overhead Principle**: *"Bạn không phải trả giá cho những gì bạn không sử dụng"*.
Do đó, trong C++, mọi hàm thành viên mặc định **100% là Static Binding**. Chỉ khi lập trình viên chủ động gắn từ khóa `virtual`, trình biên dịch mới kích hoạt cơ chế Dynamic Binding.

5.2 Hàm Ảo, `override` và `final` trong C++11

5.2.1 Cơ chế Hàm Ảo (`virtual`)

Khi khai báo một phương thức với từ khóa `virtual` ở lớp cơ sở, ta đã thiết lập một giao ước đa hình:

```
class Shape {
public:
    virtual void draw() const {
        std::cout << "Drawing generic shape\n";
    }
    virtual ~Shape() = default;
};

class Circle : public Shape {
public:
    void draw() const override { // Override virtual method
        std::cout << "Drawing Circle\n";
    }
};
```

Quy tắc lan truyền ảo (Virtual Propagation): Một khi một phương thức đã được khai báo là `virtual` ở lớp cơ sở, nó sẽ **tự động mang tính ảo vĩnh viễn** trong toàn bộ cây thừa kế của các lớp con cháu, cho dù ở lớp con ta có viết lại từ khóa `virtual` hay không.

5.2.2 Bộ Đôi Từ Khóa Ngữ Cảnh C++11: `override` và `final`

Trước C++11, lỗi ngầm phổ biến nhất của lập trình viên là viết sai chữ ký hàm con dẫn đến việc vô tình tạo ra một hàm nạp chồng mới thay vì ghi đè hàm cha. Chuẩn C++11 bổ sung hai từ khóa ngữ cảnh (*contextual keywords*):

- **`override`**: Yêu cầu trình biên dịch kiểm tra nghiêm ngặt xem hàm này có thực sự ghi đè một hàm ảo nào từ lớp cha hay không. Nếu chữ ký hàm, kiểu trả về, hoặc thuộc tính `const` không khớp 100%, trình biên dịch lập tức báo lỗi biên dịch (*Compilation Error*):

```
    class DerivedC11 : public Base {
    public:
        void process(int x) const override; // Hop le: Khop chinh xac chu ky
     cua Base
    };

```

- **`final`**: Khóa không cho lớp con tiếp tục ghi đè phương thức ảo hoặc khóa hoàn toàn không cho phép kế thừa lớp:

```
    virtual void process(int x) const override final; // Chan ghi de o lop
    chau
    class SecureKey final : public Base { /* ... */ }; // Chan ke thua

```

Tối ưu hóa Bỏ ảo hóa (Devirtualization): Khi trình biên dịch phát hiện một phương thức hoặc lớp được đánh dấu `final`, nó có thể biến đổi các lời gọi gián tiếp thành lệnh gọi trực tiếp (**Direct Call**), loại bỏ hoàn toàn chi phí tra cứu bảng ảo.

5.3 Phân tích Dưới Gầm Máy: Bảng Ảo (VTable) & Con trỏ vptr

Làm thế nào mà CPU có thể tìm ra đúng hàm `Circle::draw()` khi nó chỉ đang giữ một con trỏ kiểu `Shape*`? Câu trả lời nằm ở cấu trúc nhị phân của Bảng Phương Thức Ảo:

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Kiến Trúc Bảng Ảo VTable và Con Trỏ vptr Dưới RAM

1. **Virtual Method Table (VTable):** Là mảng tĩnh các con trỏ hàm được sinh ra duy nhất một bản cho mỗi lớp có phương thức ảo, lưu tại phân vùng chỉ đọc (`.rodata` trên Linux ELF hoặc `.rdata` trên Windows PE).

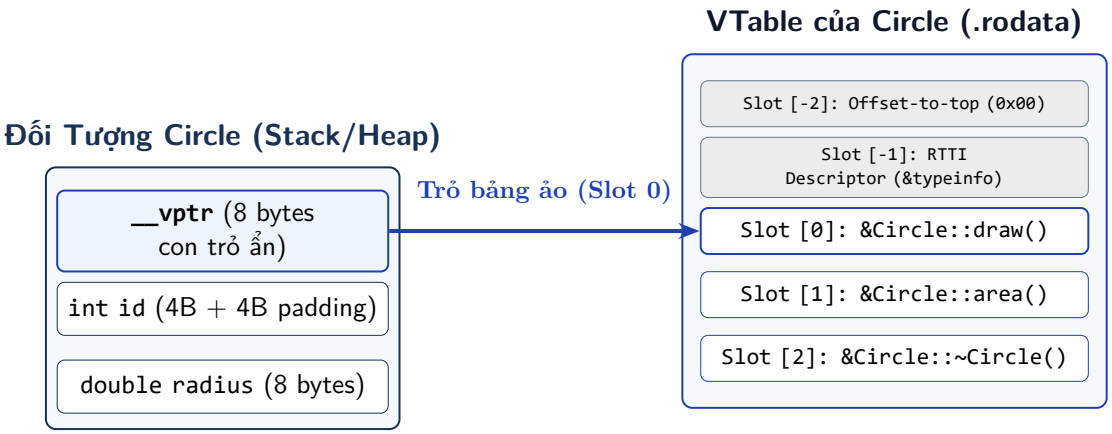


Figure 5.1: Cấu trúc nhị phân của Bảng Phương Thức Ảo (VTable) bao gồm các slot tiền tố âm RTTI.

2. **Tiền tố offset âm (Negative Prefix Slots):** Phía trước vị trí con trỏ trỏ tới (offset 0), VTable chứa hai trường thông tin cực kỳ quan trọng:
   - **Offset-to-top (Offset -16 bytes):** Khoảng cách byte từ subobject hiện tại tới đỉnh của đối tượng toàn vẹn, phục vụ cho việc ép kiểu ngược trong đa kế thừa.
   - **RTTI Descriptor Pointer (Offset -8 bytes):** Con trỏ trỏ tới cấu trúc `std::type_info`, phục vụ cho toán tử `typeid` và `dynamic_cast`.
3. **Virtual Table Pointer (vptr):** Con trỏ ẩn 8 bytes được chèn vào đầu đối tượng, trỏ thẳng tới Slot [0] của VTable, được thiết lập tự động bởi mã máy của Constructor.

Giải phẫu Lệnh Nhảy Gián Tiếp (Indirect Dispatching): Khi thực thi `pShape->draw()`:

1. CPU lấy địa chỉ đối tượng từ thanh ghi con trỏ (ví dụ `RDI` hoặc `RCX`).
2. CPU giải tham chiếu offset 0 để lấy giá trị con trỏ `__vptr`: `RAX = *(pShape)`.
3. CPU tra cứu mảng VTable tại slot của hàm `draw()` (slot 0): `RDX = *(RAX + 0)`.
4. CPU thực hiện lệnh nhảy gián tiếp: `call RDX`, đồng thời truyền con trỏ đối tượng `pShape` làm con trỏ bản thể `this`.

5.3.1 Hàm Thunk Bù Độ Lệch Con Trỏ Bản Thể (Adjustor Thunk)

Trong Đa kế thừa, khi lớp `C` kế thừa từ cả `A` và `B`, Subobject `B` nằm lệch đi một khoảng Δ (ví dụ +8 bytes). Khi gọi một phương thức ảo của B thông qua con trỏ `B* pb = new C(); pb->fb();`, con trỏ truyền vào đang trỏ tại offset +8, trong khi thân hàm `C::fb()` kỳ vọng con trỏ `this` trỏ tại offset 0!

Giải pháp Thunk: Trình biên dịch sinh ra một đoạn mã đệm siêu nhỏ:

```
__thunk_for_C_fb:
    sub rcx, 8          ; Rewind this pointer (MSVC x64 ABI)
    ; sub rdi, 8        ; Tuong duong tren System V AMD64 ABI (Linux)
```

```
    jmp C::fb           ; Direct jump to actual implementation
```

5.4 Kiểu Trả Về Đồng Biến (Covariant Return Types)

Một trong những tính năng cao cấp và thanh lịch nhất của đa hình C++ là **Kiểu trả về đồng biến (Covariant Return Types)**: Một phương thức ghi đè ở lớp con được phép có kiểu trả về là **con trỏ hoặc tham chiếu tới lớp con**, thay vì phải giữ nguyên kiểu của lớp cha.

```cpp
class Entity {
public:
    virtual Entity* clone() const = 0; // Pure virtual clone
    virtual ~Entity() = default;
};

class Monster : public Entity {
public:
    // Covariant Return Type: Returns Monster* instead of Entity*!
    Monster* clone() const override {
        return new Monster(*this);
    }
};

void testCloning() {
    Monster original;
    // Khong can ep kieu static_cast<Monster*>!
    Monster* copyMonster = original.clone();
    delete copyMonster;
}
```

Đây là nền tảng của thành ngữ thiết kế **Virtual Copy Constructor (Clone Idiom)**, cho phép nhân bản đối tượng đa hình một cách hoàn toàn an toàn về kiểu mà không cần ép kiểu thủ công.

5.5 Destructor Ảo (Virtual Destructor) & Bẫy Gọi Hàm Ảo Trong Constructor

5.5.1 Destructor Ảo — Nguyên Lý Sống Còn Trong Kế Thừa

```cpp
class BaseResource {
public:
    virtual ~BaseResource() = default; // Mandatory virtual destructor!
};

class DerivedResource : public BaseResource {
private:
    int* largeBuffer;
public:
    DerivedResource() : largeBuffer(new int[1000000]) {}
    ~DerivedResource() override {
        delete[] largeBuffer; // Safely executed via virtual dispatch!
    }
```

```
};
```

Tiên Đề / Định Nghĩa Pháp Lập: Quy Tắc Vàng Của Scott Meyers Về Destructor

- **Quy tắc 1:** Bất kỳ lớp nào được thiết kế để làm lớp cơ sở có tính đa hình (*Polymorphic Base Class*), Destructor của nó **BẮT BUỘC PHẢI LÀ** `public virtual`.
- **Quy tắc 2:** Nếu một lớp không được thiết kế để làm lớp cha đa hình, hãy khai báo Destructor là `protected non-virtual`. Cách này ngăn chặn việc gọi `delete ptrBase` từ bên ngoài, triệt tiêu lỗi rò rỉ ngay tại compile-time.

5.5.2 Bẫy Gọi Hàm Ảo Trong Constructor & Destructor

Trong C++, gọi một hàm ảo bên trong Constructor hoặc Destructor sẽ **chỉ kích hoạt phiên bản của chính lớp đang thực thi Constructor đó**, hoàn toàn không kích hoạt phiên bản ghi đè của lớp con!

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Nguyên Nhân Gầm Máy: Sự Tiến Hóa Của vptr Trong Vòng Đời Đối Tượng

1. Khi Constructor của lớp cha đang thực thi, phần dữ liệu của lớp con **chưa hề được cấp phát và khởi tạo**.
2. Do đó, con trỏ `__vptr` lúc này **đang trỏ tới VTable của lớp cha**, chưa trỏ tới VTable của lớp con.
3. Với các lời gọi trực tiếp (*Direct calls*), trình biên dịch tối ưu hóa tĩnh (*Static Devirtualization*) gọi thẳng hàm cha.
4. Với các lời gọi gián tiếp qua hàm bổ trợ mà hàm ảo là thuần ảo (`= 0`), hệ thống sẽ gọi con trỏ lỗi `__cxa_pure_virtual` và đình chỉ tiến trình ngay lập tức.

CHƯƠNG 6

Khuôn mẫu & Lập trình Tổng quát (Templates & Generic Programming)

Tiên Đề / Định Nghĩa Pháp Lập: Định Nghĩa: Lập Trình Tổng Quát (Generic Programming)

Lập trình Tổng quát (Generic Programming) là hệ hình lập trình trong đó mã nguồn được viết một cách trừu tượng hóa, độc lập hoàn toàn với các kiểu dữ liệu cụ thể. Thay vì phải sao chép thuật toán tìm kiếm hay cấu trúc danh sách liên kết cho từng kiểu `int`, `double`, `Student`, lập trình viên sử dụng **Khuôn mẫu (Templates)** để biến **Kiểu dữ liệu thành Tham số (Type-as-a-Parameter)**.

6.1 Khuôn Mẫu Hàm & Khuôn Mẫu Lớp (Function & Class Templates)

```
// Generic swap function template
template <typename T>
void genericSwap(T& a, T& b) {
    T temp = std::move(a);
    a = std::move(b);
    b = std::move(temp);
}

// Generic safe stack class template
template <typename T>
class SafeStack {
private:
    std::vector<T> elements;
public:
    void push(const T& val) { elements.push_back(val); }
```

```
    void push(T&& val) { elements.push_back(std::move(val)); }
    T pop() {
        if (elements.empty()) throw std::out_of_range("Stack empty");
        T val = std::move(elements.back());
        elements.pop_back();
        return val;
    }
};
```

6.2 Cơ Chế Phân Giải Tên Hai Giai Đoạn (Two-Phase Name Lookup)

Một trong những cơ chế phức tạp nhất của trình biên dịch C++ khi xử lý template là quy trình phân giải tên hai giai đoạn:

1. **Giai đoạn 1 (Phân tích cú pháp định nghĩa — Parsing Phase):** Xảy ra khi trình biên dịch lần đầu tiên quét qua khai báo khuôn mẫu. Trình biên dịch kiểm tra tính hợp lệ cú pháp của các thành phần không phụ thuộc vào tham số kiểu T.
2. **Giai đoạn 2 (Tạo thực thể — Instantiation Phase):** Xảy ra khi lập trình viên thực sự sử dụng template với kiểu cụ thể (ví dụ `SafeStack<int> s;`). Trình biên dịch thay thế tham số kiểu và sinh mã máy thực tế.

6.2.1 Từ Khóa `typename` và `.template` Cho Tên Phụ Thuộc (Dependent Names)

```
template <typename Container>
void printFirstElement(const Container& c) {
    // BAT BUOC CO 'typename' vi const_iterator la kieu phu thuoc:
    typename Container::const_iterator it = c.begin();
    if (it != c.end()) {
        std::cout << *it << "\n";
    }
}

template <typename T>
void parseData(T& parser) {
    // BAT BUOC CO '.template' de compiler khong nham '<' thanh toan tu so sanh:
    parser.template parse<int>();
}
```

6.3 Chuyên Biệt Hóa Khuôn Mẫu & Giải Pháp Hiện Đại C++17

6.3.1 Chuyên Biệt Hóa Toàn Phần (Full Specialization)

```
// 1. Primary Template
template <typename T>
struct Serializer {
```

```
    static void serialize(const T& val) { /* Generic serialization */ }
};

// 2. Full Specialization for bool:
template <>
struct Serializer<bool> {
    static void serialize(const bool& val) { /* Bit-packing optimization */ }
};
```

6.3.2 C++17 `if constexpr`: Chấm Dứt Bẫy Nạp Chồng Herb Sutter

Trước C++17, việc nạp chồng hoặc chuyên biệt hóa hàm template thường dẫn đến các bẫy phân giải phức tạp theo phân tích của Herb Sutter. Chuẩn C++17 giới thiệu **`if constexpr`**, cho phép rẽ nhánh điều kiện ngay tại thời điểm biên dịch một cách thanh lịch:

```
#include <type_traits>

template <typename T>
void processEntity(T val) {
    if constexpr (std::is_pointer_v<T>) {
        if (val) std::cout << "Pointer value: " << *val << "\n";
    } else {
        std::cout << "Direct value: " << val << "\n";
    }
}
```

Nhánh không thỏa mãn điều kiện sẽ bị loại bỏ hoàn toàn khỏi mã máy mà không cần viết các lớp chuyên biệt hóa phụ trợ!

6.4 Các Tính Năng Khuôn Mẫu Hiện Đại: Variadics & Fold Expressions

6.4.1 Biểu Thức Gập C++17 (Fold Expressions)

Từ chuẩn C++17, kỹ thuật đệ quy gói tham số phức tạp của C++11 được thay thế bằng **Biểu Thức Gập (Fold Expressions)**:

```
// C++17 Binary Right Fold: Zero recursion, zero base cases!
template <typename... Args>
void printAll(const Args&... args) {
    ((std::cout << args << " "), ...) << "\n";
}

template <typename... Args>
auto sumAll(Args... args) {
    return (... + args); // Unary left fold over + operator
}
```

6.5 CRTP: Đa Hình Tĩnh (Static Polymorphism)

Làm thế nào để đạt được sức mạnh của Đa hình nhưng **không phải trả giá cho VTable, con trỏ vptr, và indirect jump**? Câu trả lời là kỹ thuật **Curiously Recurring Template Pattern (CRTP)**:

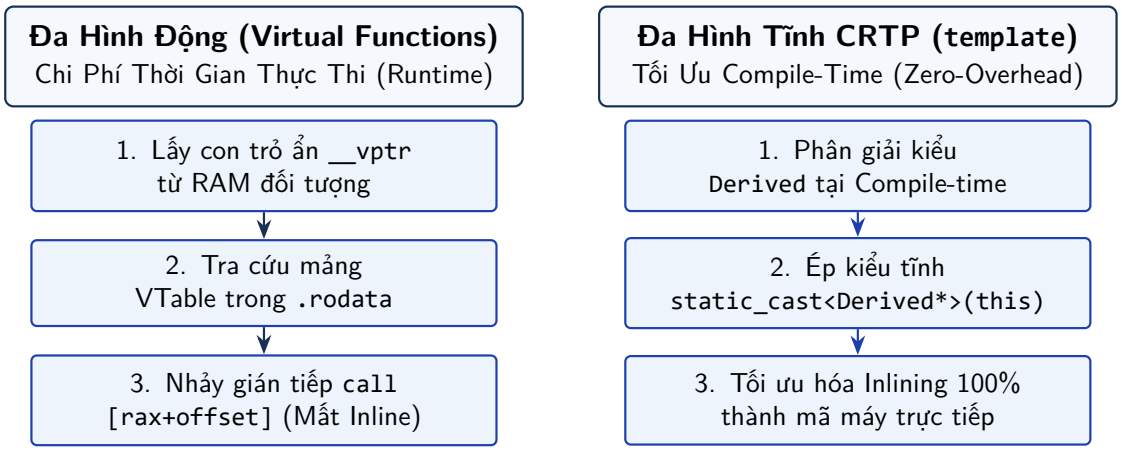


Figure 6.1: So sánh cơ chế phân giải giữa Đa hình động (VTable) và Đa hình tĩnh CRTP.

```
template <typename Derived>
class BaseProcessor {
public:
    void execute() {
        // Ep kieu tinh con tro ban the tai compile-time:
        static_cast<Derived*>(this)->processImpl();
    }
};

class FastNetworkProcessor : public BaseProcessor<FastNetworkProcessor> {
public:
    void processImpl() { // Khong can tu khoa virtual!
        std::cout << "High-speed zero-overhead processing\n";
    }
};

template <typename T>
void run(BaseProcessor<T>& proc) {
    proc.execute(); // Inlined 100%, toc do toi da!
}
```

CHƯƠNG 7

Xử lý Ngoại lệ & Kiến trúc Thư viện Chuẩn STL (Exceptions & STL)

Tiên Đề / Định Nghĩa Pháp Lập: Triết Lý Xử Lý Lỗi Hiện Đại

Trong kỹ thuật lập trình hướng thủ tục cổ điển, việc quản lý lỗi dựa hoàn toàn vào **Mã lỗi trả về (Return Error Codes)** và biến toàn cục `errno`. Cách tiếp cận này bộc lộ ba khiếm khuyết mang tính bản chất:

1. **Nguy cơ bỏ quên lỗi:** Lập trình viên dễ dàng bỏ qua việc kiểm tra giá trị trả về của hàm, khiến trạng thái lỗi lan truyền âm thầm và phá hủy dữ liệu hệ thống ở những bước tiếp theo.
2. **Ô nhiễm chữ ký hàm:** Mã lỗi làm trộn lẫn giá trị trả về hợp lệ với trạng thái thất thường, làm phức tạp hóa cấu trúc logic của chương trình (*Happy Path* bị chìm ngập trong các khối `if (err != 0)`).
3. **Sự bất lực trong Constructor: Constructor hoàn toàn không có kiểu trả về**! Do đó, mã lỗi hoàn toàn bất lực trong việc thông báo thất bại khi khởi tạo một đối tượng.

Cơ chế Ngoại lệ (Exception Handling) tách bạch triệt để luồng thực thi nghiệp vụ bình thường ra khỏi luồng cứu vãn sự cố (*Error Recovery Path*), tạo tiền đề cho các hệ thống phần mềm có độ tin cậy cấp công nghiệp.

7.1 Cơ Chế Bắt Ngoại Lệ & Tránh Cắt Xén Đối Tượng

Khi một đoạn mã phát hiện một tình huống bất thường vượt quá thẩm quyền xử lý nội bộ, nó kích hoạt ngoại lệ bằng lệnh `throw`:

```
#include <iostream>
#include <stdexcept>
```

```cpp
#include <string>

void executeTransaction(int balance, int withdrawalAmount) {
    if (withdrawalAmount > balance) {
        throw std::runtime_error("Insufficient funds: Overdraft limit exceeded");
    }
    std::cout << "[Banking] Withdrawal successful. Remaining: " << (balance - withdrawalAmount) << "\n";
}

void processAccount() {
    try {
        executeTransaction(500, 1000);
    }
    catch (const std::exception& e) { // STANDARD: Catch by const reference!
        std::cerr << "[Exception Handler] Audit log: " << e.what() << "\n";
    }
}
```

Hành Vi Không Xác Định (Undefined Behavior): Quy Tắc Bắt Ngoại Lệ Bằng Tham Chiếu Hằng

Nếu bắt ngoại lệ theo giá trị (`catch (std::exception e)`), hiện tượng **Cắt xén đối tượng đa hình (Polymorphic Object Slicing)** sẽ xảy ra: toàn bộ thông điệp và kiểu lỗi của lớp con (`std::runtime_error`) bị gọt bỏ, chỉ còn lại vỏ bọc rỗng của `std::exception`.
Quy tắc Vàng: Luôn luôn bắt ngoại lệ bằng **Tham Chiếu Hằng** (`const ExceptionType& e`).

7.2 Trải Cuộn Ngăn Xếp (Stack Unwinding) & Liên Minh RAII

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Cơ Chế Stack Unwinding & Bảng Ngoại Lệ Zero-Cost

Trong các hệ thống C++ hiện đại (tuân thủ Itanium C++ ABI trên Linux hoặc Structured Exception Handling trên Windows), trình biên dịch áp dụng mô hình **Xử lý ngoại lệ Zero-Cost (Table-Driven Zero-Cost Exception Handling)**:

1. **Tại thời điểm bình thường (Zero Runtime Overhead):** Không có bất kỳ chi phí thực thi nào cho các khối `try`. Mã máy chạy với tốc độ tối đa 100%.
2. **Khi lệnh `throw` kích hoạt ngoại lệ:**
   - Hệ thống thực hiện **Pha 1 (Search Phase):** Quét ngược Call Stack dựa trên bảng thông tin khung hàm **Call Frame Information (DWARF CFI trong phân vùng `.eh_frame`)** để tìm khối `catch` hợp lệ mà *chưa thu hồi tài nguyên.*

- Nếu tìm thấy khối catch hợp lệ, hệ thống thực hiện **Pha 2 (Cleanup / Unwind Phase):** Quét ngược lại Call Stack lần thứ hai. Tại mỗi khung hàm bị thoát ra, **MỌI ĐỐI TƯỢNG ĐỊA PHƯƠNG TRÊN STACK ĐỀU ĐƯỢC TỰ ĐỘNG GỌI DESTRUCTOR** theo thứ tự LIFO!

3. **Liên minh hoàn hảo với RAII:** Nếu toàn bộ tài nguyên được bọc trong các đối tượng RAII (`std::unique_ptr`, `std::ofstream`, `std::lock_guard`), thì cho dù ngoại lệ có bay qua 10 tầng hàm, toàn bộ bộ nhớ và file đều được đóng sạch sẽ 100%.

7.3 Đặc Tả `noexcept` & Tối Ưu Hóa Tái Cấp Phát `std::vector`

```
void hardwareReset() noexcept {
    // Cam ket khong bao gio de ngoai le thoat ra ngoai
}
```

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Toán Tử `noexcept(expr)` Và Chiến Lược Tái Cấp Phát Của `std::vector`

Xét kịch bản `std::vector<Widget>` bị đầy dung lượng và phải cấp phát một vùng nhớ mới có dung lượng gấp đôi trên Heap:

1. Để đáp ứng **Cam Kết An Toàn Ngoại Lệ Mạnh Mẽ (Strong Exception Guarantee)**: Nếu có lỗi xảy ra trong quá trình di dời dữ liệu, trạng thái của vector cũ phải được bảo toàn nguyên vẹn 100% như trước khi gọi hàm.
2. Trình biên dịch sử dụng toán tử compile-time: `noexcept(Widget(std::declval<Widget&&>()))`.
3. **Move Constructor CÓ `noexcept`:** Vector sử dụng Move Constructor để di dời từng phần tử sang vùng nhớ mới với chi phí hoán đổi con trỏ $O(1)$.
4. **Move Constructor THIẾU `noexcept`:** Vector lo ngại ngoại lệ xảy ra giữa chừng sẽ làm hỏng dữ liệu gốc không thể khôi phục được. Do đó, **Vector BẮT BUỘC PHẢI DÙNG COPY CONSTRUCTOR**! Toàn bộ dữ liệu bị sao chép sâu chậm chạp, triệt tiêu lợi thế tốc độ của C++11.

7.4 Ba Cấp Độ Cam Kết An Toàn Ngoại Lệ (Exception Safety Guarantees)

Một hệ thống phần mềm C++ chuẩn mực phải phân định rõ ba cấp độ an toàn ngoại lệ theo phân loại của David Abrahams:

1. **Cam Kết Cơ Bản (Basic Guarantee):** Nếu một ngoại lệ xảy ra, hệ thống bảo đảm **không có tài nguyên nào bị rò rỉ** và tất cả các đối tượng đều ở trạng thái hợp lệ nội tại (*Valid Invariants*).

2. **Cam Kết Mạnh Mẽ (Strong Guarantee / Commit-or-Rollback):** Thao tác mang tính nguyên tử (*Atomic*): Hoặc là thành công trọn vẹn, hoặc nếu có ngoại lệ thì trạng thái của toàn bộ hệ thống được phục hồi nguyên vẹn 100% như trước khi gọi hàm (Copy-and-Swap).
3. **Cam Kết Không Bao Giờ Thất Bại (Nothrow / No-fail Guarantee):** Hàm cam đoan 100% thành công và không bao giờ ném ra bất kỳ ngoại lệ nào (Mọi Destructor, hàm giải phóng tài nguyên, và Move Constructors).

Khảo Sát Kiến Trúc Phần Cứng & Trình Biên Dịch: Lý Do Thiết Kế Đằng Sau `std::stack::pop()`

Tại sao trong STL, hàm `std::stack::pop()` chỉ xóa phần tử đỉnh mà trả về kiểu `void`?

- Nếu hàm `T pop()` trả về theo giá trị, sau khi phần tử đã bị xóa khỏi Stack, lệnh `return result;` kích hoạt Copy Constructor của kiểu `T`.
- Nếu phép sao chép ném ra ngoại lệ `std::bad_alloc`: Phần tử đã bị xóa mất khỏi Stack nhưng hàm gọi chưa hề nhận được dữ liệu! Dữ liệu bị **mất trắng vĩnh viễn**, phá vỡ Cam Kết An Toàn Mạnh Mẽ!
- Bằng cách tách biệt `top()` (chỉ đọc tham chiếu) và `pop()` (xóa an toàn `noexcept`), STL bảo đảm trọn vẹn tính toàn vẹn dữ liệu.

7.5 Kiến Trúc Tách Rời Trong Thư Viện Chuẩn C++ (STL)

Thư viện Chuẩn C++ (**Standard Template Library** — **STL**) do Alexander Stepanov sáng tạo là kiệt tác của sự kết hợp giữa Lập trình Tổng quát và Tư duy Hướng Đối tượng.

7.5.1 Kiến Trúc Ba Tầng Tách Rời (Decoupled Architecture)



Figure 7.1: Kiến trúc ba tầng tách rời (Decoupled Architecture) của Alexander Stepanov trong STL.

Bằng cách phân tách hoàn toàn Thuật toán và Cấu trúc dữ liệu thông qua giao diện trung gian là **Bộ lặp (Iterators)**, N cấu trúc dữ liệu và M thuật toán chỉ đòi hỏi đúng $N + M$ thành phần độc lập thay vì $N \times M$ phương thức!

7.5.2 Kỹ Thuật Tag Dispatching Tại Compile-Time

```cpp
// Optimal implementation for Random Access Iterator in O(1)
template <typename RandomAccessIterator, typename Distance>
void advance_impl(RandomAccessIterator& it, Distance n, std::
    random_access_iterator_tag) {
    it += n;
}

// Fallback implementation for Bidirectional Iterator in O(N)
template <typename BidirectionalIterator, typename Distance>
void advance_impl(BidirectionalIterator& it, Distance n, std::
    bidirectional_iterator_tag) {
    while (n > 0) { ++it; --n; }
    while (n < 0) { --it; ++n; }
}

template <typename Iterator, typename Distance>
void advance(Iterator& it, Distance n) {
    advance_impl(it, n, typename std::iterator_traits<Iterator>::iterator_category
    ());
}
```

7.5.3 Bộ Cấp Phát Bộ Nhớ Tùy Biến (Allocators & Rebind Mechanism)

Khi `std::list<T>` cần cấp phát bộ nhớ cho nút `ListNode<T>` thay vì kiểu `T` đơn thuần, nó sử dụng **Cơ Chế Tái Liên Kết (The Rebind Mechanism)**:

```cpp
template <typename T, typename Allocator = std::allocator<T>>
class CustomList {
    // Rebind Allocator from type T to ListNode<T>:
    using NodeAlloc = typename std::allocator_traits<Allocator>::template
    rebind_alloc<ListNode<T>>;
    NodeAlloc nodeAllocator;
};
```

CHƯƠNG 8

Nguyên lý Thiết kế SOLID & Mẫu Kiến trúc Hiện đại (SOLID & Design Patterns)

Tiên Đề / Định Nghĩa Pháp Lập: Bản Chất Của Thiết Kế Kiến Trúc Hướng Đối Tượng

Một chương trình C++ có thể biên dịch thành công và quản lý bộ nhớ không bị rò rỉ, nhưng vẫn có thể là một phần mềm thất bại nếu kiến trúc của nó giòn vỡ (*Brittle*), cứng nhắc (*Rigid*), và khó kiểm thử (*Untestable*).
Thiết kế Hướng Đối tượng chuẩn mực nhằm tối ưu hóa hai thước đo nền tảng của khoa học kỹ nghệ phần mềm:

1. **Độ Kết Dính Cao (High Cohesion):** Mỗi mô-đun, lớp, hoặc hàm chỉ tập trung hiện thực hóa một chức năng chuyên biệt duy nhất. Mọi biến thành viên và phương thức đều phục vụ một mục đích cốt lõi nhất quán.
2. **Độ Liên Kết Thấp (Low Coupling):** Các lớp trong hệ thống hạn chế tối đa sự phụ thuộc trực tiếp vào cấu trúc cài đặt cụ thể của nhau, giao tiếp thuần túy qua các giao diện trừu tượng (*Abstract Interfaces*).

8.1 Hệ Thống Năm Nguyên Lý Thiết Kế SOLID

Bộ năm nguyên lý SOLID do Robert C. Martin tổng hợp là bộ quy tắc vàng cho mọi kỹ sư phần mềm chuyên nghiệp:

8.1.1 1. Single Responsibility Principle (Nguyên Lý Đơn Trách Nhiệm - SRP)

"Một lớp chỉ nên có duy nhất một lý do để thay đổi."

```
1 // VI PHAM SRP: Lop vua quan ly du lieu vua phu trach ghi file va gui mang
```

```cpp
class UserProfileBad {
    std::string username;
public:
    void changePassword(const std::string& newPass);
    void saveToDatabase(); // Trach nhiem luu tru
    void sendWelcomeEmail(); // Trach nhiem giao tiep mang
};

// TUAN THU SRP: Tach biet ro rang ba pham vi trach nhiem
class UserProfile {
    std::string username;
public:
    void changePassword(const std::string& newPass);
};

class UserRepository {
public:
    void save(const UserProfile& user); // Chuyen trach luu tru
};

class NotificationService {
public:
    void sendEmail(const UserProfile& user, const std::string& message); // Chuyen
     trach truyen thong
};
```

8.1.2 2. Open/Closed Principle (Nguyên Lý Đóng/Mở - OCP)

"Thực thể phần mềm phải mở cho việc mở rộng (Open for extension), nhưng đóng đối với việc sửa đổi (Closed for modification)."

Thay vì dùng chuỗi lệnh `switch-case` hay `if-else` kiểm tra kiểu để thực hiện hành vi, ta sử dụng tính Đa hình (*Polymorphism*) để thêm hành vi mới bằng cách tạo thêm lớp phái sinh mà không sửa một dòng code cũ nào!

8.1.3 3. Liskov Substitution Principle (Nguyên Lý Thay Thế Liskov - LSP)

"Các đối tượng của lớp con phải có khả năng thay thế hoàn toàn cho các đối tượng của lớp cha mà không làm phá vỡ tính đúng đắn của chương trình."

Hành Vi Không Xác Định (Undefined Behavior): Nghịch Lý Hình Vuông — Hình Chữ Nhật (The Rectangle-Square Dilemma)

Trong toán học hình học, *"Hình vuông là một Hình chữ nhật đặc biệt"*. Tuy nhiên, mô hình hóa mối quan hệ này bằng kế thừa trong C++ là một sai lầm chết người:

```cpp
class Rectangle {
protected:
    double width, height;
public:
```

```
    virtual void setWidth(double w) { width = w; }
    virtual void setHeight(double h) { height = h; }
    double getArea() const { return width * height; }
};

class Square : public Rectangle {
public:
    void setWidth(double w) override { width = height = w; }
    void setHeight(double h) override { width = height = h; }
};

void clientFunction(Rectangle& r) {
    r.setWidth(5);
    r.setHeight(4);
    // Voi Rectangle: Dien tich phai la 20!
    // Voi Square: r.setHeight(4) lam width bien thanh 4 -> Dien tich = 16!
    assert(r.getArea() == 20); // THAT BAI VOI SQUARE! VI PHAM LSP!
}
```

Hệ luận kiến trúc: Quan hệ kế thừa trong lập trình không chỉ yêu cầu tính tương thích về mặt từ vựng (*IS-A*), mà đòi hỏi sự tương thích tuyệt đối về **Hành vi và Điều kiện bất biến (*Behavioral Subtyping*)**.

8.1.4 4. Interface Segregation Principle (Nguyên Lý Phân Tách Giao Diện - ISP)

"Khách hàng không nên bị ép buộc phải phụ thuộc vào các giao diện mà họ không sử dụng."

Thay vì tạo ra một giao diện "toàn năng" khổng lồ với 50 phương thức ảo, hãy chia nhỏ thành nhiều giao diện chuyên biệt, gọn gàng.

8.1.5 5. Dependency Inversion Principle (Nguyên Lý Đảo Ngược Phụ Thuộc - DIP)

"Các mô-đun cấp cao không nên phụ thuộc vào các mô-đun cấp thấp. Cả hai phải cùng phụ thuộc vào sự trừu tượng hóa (Abstractions)."

8.2 Các Mẫu Thiết Kế GoF Hiện Đại Hóa Trong C++11/14/17

8.2.1 Meyers Singleton (Thread-Safe Static Local Initialization)

Trong C++98, mẫu Singleton đòi hỏi kỹ thuật khóa hai lần phức tạp (*Double-Checked Locking Pattern*) và dễ vỡ do tái sắp xếp lệnh của CPU. Từ C++11, điều khoản [stmt.dcl] p4 quy định: Việc khởi tạo biến tĩnh cục bộ được bảo đảm **An toàn Đa luồng 100% bởi Hệ điều hành**:

```cpp
class SystemLogger {
private:
    SystemLogger() = default; // Private constructor
    ~SystemLogger() = default;
    SystemLogger(const SystemLogger&) = delete;
    SystemLogger& operator=(const SystemLogger&) = delete;

public:
    static SystemLogger& getInstance() {
        // Thread-safe dynamic initialization guaranteed by C++11 magic statics:
        static SystemLogger instance;
        return instance;
    }

    void log(const std::string& msg) {
        std::cout << "[SYSTEM AUDIT] " << msg << "\n";
    }
};
```

8.2.2 Mẫu Nhà Máy (Factory Method Pattern) Kết Hợp Smart Pointers

```cpp
#include <memory>

class IProtocolHandler {
public:
    virtual void send(const std::string& payload) = 0;
    virtual ~IProtocolHandler() = default;
};

class TcpHandler : public IProtocolHandler {
public:
    void send(const std::string& payload) override { /* TCP logic */ }
};

class UdpHandler : public IProtocolHandler {
public:
    void send(const std::string& payload) override { /* UDP logic */ }
};

// Modern Factory Method returning exclusive ownership:
std::unique_ptr<IProtocolHandler> createHandler(const std::string& type) {
    if (type == "TCP") return std::make_unique<TcpHandler>();
    if (type == "UDP") return std::make_unique<UdpHandler>();
    return nullptr;
}
```

8.2.3 Mẫu Chiến Lược (Strategy Pattern): Đa Hình Động vs Đa Hình Tĩnh

Mẫu Chiến lược cho phép hoán đổi thuật toán linh hoạt tại runtime hoặc compile-time:

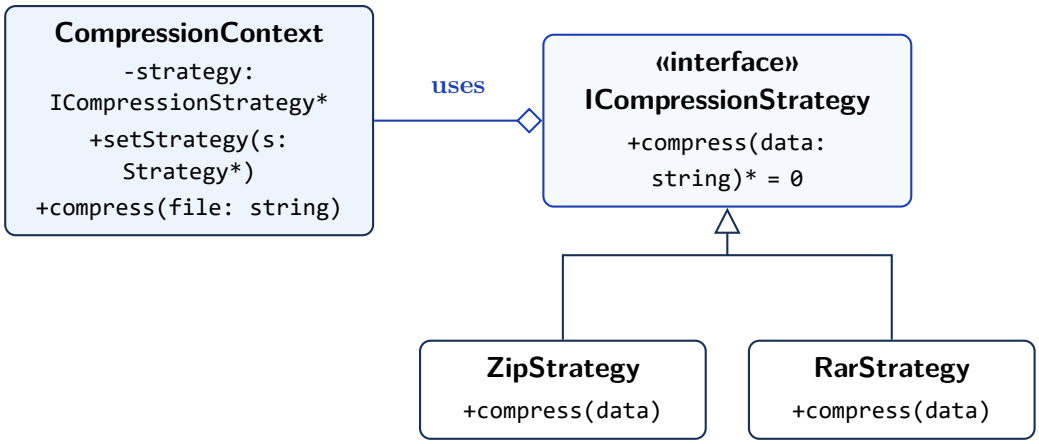


Figure 8.1: Biểu đồ lớp UML chuẩn của Mẫu Thiết Kế Chiến Lược (Strategy Pattern).

Lời Kết: Con Đường Làm Chủ C++ Hiện Đại

Trải qua 8 chương sách kinh điển của **"The OOP C++ Bible"**, chúng ta đã cùng nhau du hành từ tầng kiến trúc bộ nhớ vật lý thấp nhất (Stack, Heap, Cache, Calling Conventions, ABI Assembly Dispatching, VTable) lên đến những tầng trừu tượng thanh lịch nhất của khoa học máy tính hiện đại (RAII, Move Semantics, Variadic Templates, CRTP, Exception Safety Guarantees, và Bộ nguyên lý SOLID).

C++ không phải là một ngôn ngữ dễ dãi. Nó không che giấu sự phức tạp của phần cứng sau một máy ảo chậm chạp. Nó trao toàn bộ quyền kiểm soát tối thượng vào tay lập trình viên, và như Bjarne Stroustrup từng đúc kết:

"C makes it easy to shoot yourself in the foot; C++ makes it harder, but when you do, it blows your whole leg off."

Nhưng khi bạn đã nắm vững các nguyên lý "dưới gầm máy" được trình bày trong cuốn sách này, bạn không còn sợ hãi những cạm bẫy ngầm của bộ nhớ nữa. Bạn đã nắm trong tay sức mạnh để kiến tạo nên những hệ thống phần mềm có hiệu năng vô song, an toàn tuyệt đối và trường tồn cùng thời gian.

Chúc các bạn luôn giữ vững niềm đam mê và tự hào trên con đường trở thành một **C++ Master**!

Bibliography

[1] ISO/IEC 14882:2011, *Information technology — Programming languages — C++*, International Organization for Standardization, Geneva, Switzerland.

[2] ISO/IEC 14882:2014, *Information technology — Programming languages — C++*, International Organization for Standardization, Geneva, Switzerland.

[3] ISO/IEC 14882:2017, *Information technology — Programming languages — C++*, International Organization for Standardization, Geneva, Switzerland.

[4] Bjarne Stroustrup (2013), *The C++ Programming Language* (4th Edition), Addison-Wesley Professional.

[5] Bjarne Stroustrup (1994), *The Design and Evolution of C++*, Addison-Wesley Professional.

[6] Scott Meyers (2005), *Effective C++: 55 Specific Ways to Improve Your Programs and Designs* (3rd Edition), Addison-Wesley Professional.

[7] Scott Meyers (2014), *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*, O'Reilly Media.

[8] Andrei Alexandrescu (2001), *Modern C++ Design: Generic Programming and Design Patterns Applied*, Addison-Wesley Professional.

[9] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides (1994), *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley Professional.

[10] Stanley B. Lippman (1996), *Inside the C++ Object Model*, Addison-Wesley Professional.

[11] David Vandevoorde, Nicolai M. Josuttis, Douglas Gregor (2017), *C++ Templates: The Complete Guide* (2nd Edition), Addison-Wesley Professional.

[12] Codesourcery, Compaq, EDG, HP, IBM, Intel, SGI (2001–2024), *Itanium C++ ABI (Revision 1.86)*, Mentored by the LLVM and GCC Compiler Working Groups.

[13] Alexander Stepanov, Meng Lee (1995), *The Standard Template Library*, HP Laboratories Technical Report 95-11(R.1).